model. This model is based on the idea of a **task bag** full of jobs to be done. The main process starts out by executing a loop containing

```
out("task-bag", job);
```

in which a different job description is output to the tuple space on each iteration. Each worker starts out by getting a job description tuple using

```
in("task-bag", ?job);
```

which it then carries out. When it is done, it gets another. New work may also be put into the task bag during execution. In this simple way, work is dynamically divided among the workers, and each worker is kept busy all the time, all with little overhead.

In certain ways, Linda is similar to Prolog, on which it is loosely based. Both support an abstract space that functions as a kind of data base. In Prolog, the space holds facts and rules; in Linda it holds tuples. In both cases, processes can provide templates to be matched against the contents of the data base.

Despite these similarities, the two systems also differ in significant ways. Prolog was intended for programming artificial intelligence applications on a single processor, whereas Linda was intended for general programming on multicomputers. Prolog has a complex pattern-matching scheme involving unification and backtracking; Linda's matching algorithm is much simpler. In Linda, a successful match removes the matching tuple from the tuple space; in Prolog it does not. Finally, a Linda process unable to locate a needed tuple blocks, which forms the basis for interprocess synchronization. In Prolog, there are no processes and programs never block.

Implementation of Linda

Efficient implementations of Linda are possible on various kinds of hardware. Below we will discuss some of the more interesting ones. For all implementations, a preprocessor scans the Linda program, extracting useful information and converting it to the base language where need be (e.g., the string "? int" is not allowed as a parameter in C or FORTRAN). The actual work of inserting and removing tuples from the tuple space is done during execution by the Linda runtime system.

An efficient Linda implementation has to solve two problems:

1. How to simulate associative addressing without massive searching.
2. How to distribute tuples among machines and locate them later.

The key to both problems is to observe that each tuple has a **type signature**, consisting of the (ordered) list of the types of its fields. Furthermore, by

convention, the first field of each tuple is normally a string that effectively partitions the tuple space into disjoint subspaces named by the string. Splitting the tuple space into subspaces, each of whose tuples has the same type signature and same first field, simplifies programming and makes certain optimizations possible.

For example, if the first parameter to an *in* or *out* call is a literal string, it is possible to determine at compile time which subspace the call operates on. If the first parameter is a variable, the determination is made at run time. In both cases, this partitioning means that only a fraction of the tuple space has to be searched. Figure 6-37 shows four tuples and four templates. Together they form four subspaces. For each *out* or *in*, it is possible to determine at compile time which subspace and tuple server are needed. If the initial string was a variable, the determination of the correct subspace would have to be delayed until run time.

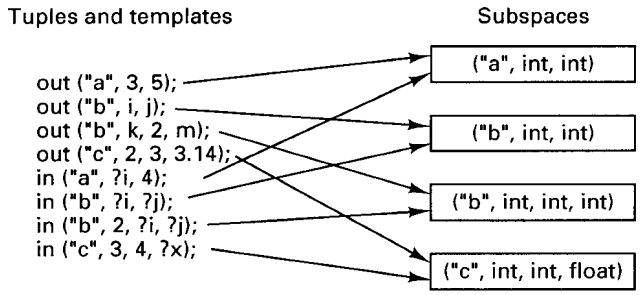


Fig. 6-37. Tuples and templates can be associated with subspaces.

In addition, each subspace can be organized as a hash table using its *i*th tuple field as the hash key. If field *i* is a constant or variable (but not a formal parameter), an *in* or *out* can be executed by computing the hash function of the *i*th field to find the position in the table where the tuple belongs. Knowing the subspace and table position eliminates all searching. If the *i*th field of a certain *in* is a formal parameter, hashing is not possible, so a complete search of the subspace is needed except in some special cases. By carefully choosing the field to hash on, however, the preprocessor can usually avoid searching most of the time. Other subspace organizations beside hashing are also possible for special cases (e.g., a queue when there is one writer and one reader).

Additional optimizations are also used. For example, the hashing scheme described above distributes the tuples of a given subspace into bins, to restrict searching to a single bin. It is possible to place different bins on different machines, both to spread the load more widely and to take advantage of locality. If the hashing function is the key modulo the number of machines, the number of bins scales linearly with the system size.

Now let us examine various implementation techniques for different kinds of hardware. On a multiprocessor, the tuple subspaces can be implemented as hash tables in global memory, one for each subspace. When an *in* or an *out* is performed, the corresponding subspace is locked, the tuple entered or removed, and the subspace unlocked.

On a multicomputer, the best choice depends on the communication architecture. If reliable broadcasting is available, a serious candidate is to replicate all the subspaces in full on all machines, as shown in Fig. 6-38. When an *out* is done, the new tuple is broadcast and entered into the appropriate subspace on each machine. To do an *in*, the local subspace is searched. However, since successful completion of an *in* requires removing the tuple from the tuple space, a delete protocol is required to remove it from all machines. To prevent races and deadlocks, a two-phase commit protocol can be used.

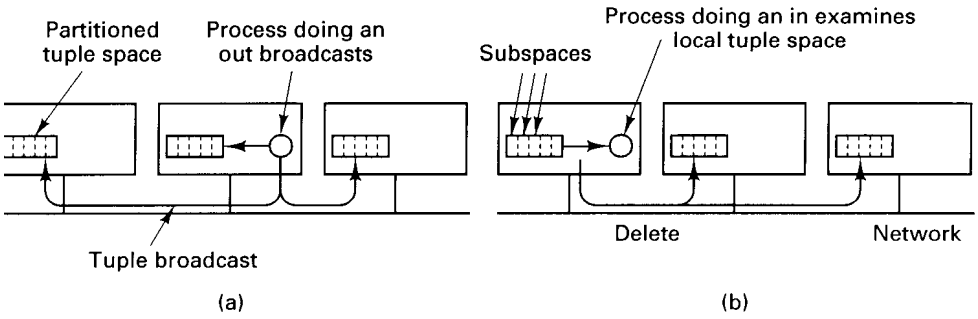


Fig. 6-38. Tuple space can be replicated on all machines. The dotted lines show the partitioning of the tuple space into subspaces. (a) Tuples are broadcast on *out*. (b). *In*s are local, but the deletes must be broadcast.

This design is straightforward, but may not scale well as the system grows in size, since every tuple must be stored on every machine. On the other hand, the total size of the tuple space is often quite modest, so problems may not arise except in huge systems. The S/Net Linda system uses this approach because S/Net has a fast, reliable, word-parallel bus broadcast (Carriero and Gelernter, 1986).

The inverse design is to do *out*s locally, storing the tuple only on the machine that generated it, as shown in Fig. 6-39. To do an *in*, a process must broadcast the template. Each recipient then checks to see if it has a match, sending back a reply if it does.

If the tuple is not present, or if the broadcast is not received at the machine holding the tuple, the requesting machine retransmits the broadcast request ad infinitem, increasing the interval between broadcasts until a suitable tuple materializes and the request can be satisfied. If two or more tuples are sent, they

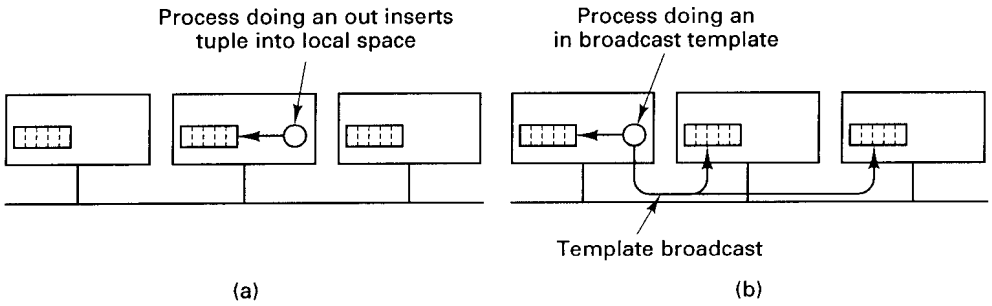


Fig. 6-39. Unreplicated tuple space. (a) An *out* is done locally. (b) An *in* requires the template to be broadcast in order to find a tuple.

are treated like local *out*s and the tuples are effectively moved from the machines that had them to the one doing the request. In fact, the runtime system can even move tuples around on its own to balance the load. Carriero et al. (1986) used this method for implementing Linda on a LAN.

These two methods can be combined to produce a system with partial replication. A simple example is to imagine all the machines logically forming a rectangle, as shown in Fig. 6-40. When a process on a machine, *A*, wants to do an *out*, it broadcasts (or sends by point-to-point message) the tuple to all machines in its row of the matrix. When a process on a machine, *B*, wants to do an *in* it broadcasts the template to all machines in its column. Due to the geometry, there will always be exactly one machine that sees both the tuple and the template (*C* in this example), and that machine makes the match and sends the tuple to the process asking for it. Krishnaswamy (1991) used this method for a hardware Linda coprocessor.

Finally, let us consider the implementation of Linda on systems that have no broadcast capability at all (Bjornson, 1993). The basic idea is to partition the tuple space into disjoint subspaces, first by creating a partition for each type signature, then by dividing each of these partitions again based on the first field. Potentially, each of the resulting partitions can go on a different machine, handled by its own tuple server, to spread the load around. When either an *out* or an *in* is done, the required partition is determined, and a single message is sent to that machine either to deposit a tuple there or to retrieve one.

Experience with Linda shows that distributed shared memory can be handled in a radically different way than moving whole pages around, as in the page-based systems we studied above. It is also quite different from sharing variables with release or entry consistency. As future systems become larger and more powerful, novel approaches such as this may lead to new insights into how to program these systems in an easier way.

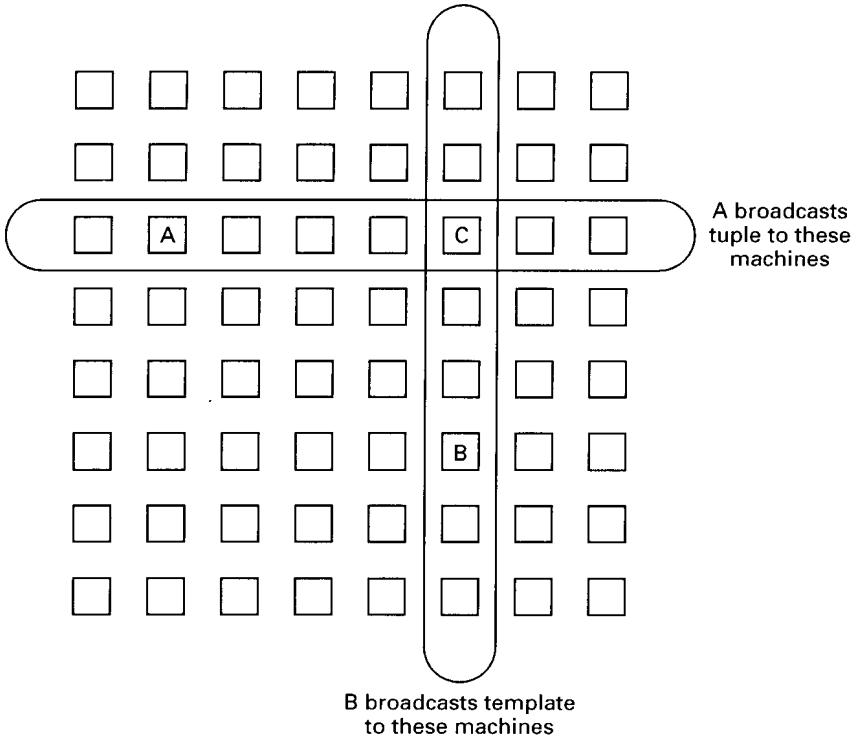


Fig. 6-40. Partial broadcasting of tuples and templates.

6.6.3. Orca

Orca is a parallel programming system that allows processes on different machines to have controlled access to a distributed shared memory consisting of protected objects (Bal, 1991; and Bal et al., 1990, 1992). These objects can be thought of as a more powerful (and more complicated) form of the Linda tuples, supporting arbitrary operations instead of just *in* and *out*. Another difference is that Linda tuples are created on-the-fly during execution in large volume, whereas Orca objects are not. The Linda tuples are used primarily for communication, whereas the Orca objects are also used for computation and are generally more heavyweight.

The Orca system consists of the language, compiler, and runtime system, which actually manages the shared objects during execution. Although language, compiler, and runtime system were designed to work together, the runtime system is independent of the compiler and could be used for other languages as well. After an introduction to the Orca language, we will describe how the runtime system implements an object-based distributed shared memory.

The Orca Language

In some respects, Orca is a traditional language whose sequential statements are based roughly on Modula-2. However, it is a type secure language with no pointers and no aliasing. Array bounds are checked at runtime (except when the checking can be done at compile time). These and similar features eliminate or detect many common programming errors such as wild stores, into memory. The language features have been chosen carefully to make a variety of optimizations easier.

Two features of Orca important for distributed programming are shared data-objects (or just **objects**) and the **fork** statement. An object is an abstract data type, somewhat analogous to a package in Ada®. It encapsulates internal data structures and user-written procedures, called **operations** (or **methods**) for operating on the internal data structures. Objects are passive, that is, they do not contain threads to which messages can be sent. Instead, processes access an object's internal data by invoking its operations. Objects do not inherit properties from other objects, so Orca is considered an object-based language rather than an object-oriented language.

Each operation consists of a list of (guard, block-of-statements) pairs. A guard is a Boolean expression that does not contain any side effects, or the empty guard, which is the same as the value *true*. When an operation is invoked, all of its guards are evaluated in an unspecified order. If all of them are *false*, the invoking process is delayed until one becomes *true*. When a guard is found that evaluates to *true*, the block of statements following it is executed. Figure 6-41 depicts a *stack* object with two operations, *push* and *pop*.

```
Object implementation stack;                    # variable indicating the top
    top: integer;                               # storage for the stack
    stack: array [integer 0..N–1] of integer;
    operation push (item: integer);             # function returning nothing
    begin
        stack [top] := item;                    # push item onto the stack
        top := top + 1;                         # increment the stack pointer
    end;
    operation pop(): integer;                   # function returning an integer
    begin
        guard top > 0 do                        # suspend if the stack is empty
            top := top – 1;                     # decrement the stack pointer
            return stack [top];                 # return the top item
        od;
    end;
begin
    top := 0;                                   # initialization
end;
```

Fig. 6-41. A simplified stack object, with internal data and two operations.

Once a *stack* has been defined, variables of this type can be declared, as in

```
s, t: stack;
```

which creates two stack objects and initializes the *top* variable in each to 0. The integer variable *k* can be pushed onto the stack *s* by the statement

```
s$push(k);
```

and so forth. The *pop* operation has a guard, so an attempt to pop a variable from an empty stack will suspend the caller until another process has pushed something on the stack.

Orca has a **fork** statement to create a new process on a user-specified processor. The new process runs the procedure named in the **fork** statement. Parameters, including objects, may be passed to the new process, which is how objects become distributed among machines. For example, the statement

```
for i in 1 .. n do fork foobar(s) on i; od;
```

generates one new process on each of machines 1 through *n*, running the process **foobar** in each of them. As these *n* new processes (and the parent) execute in parallel, they can all push and pop items onto the shared stack *s* as though they were all running on a shared-memory multiprocessor. It is the job of the runtime system to sustain the illusion of shared memory where it really does not exist.

Operations on shared objects are atomic and sequentially consistent. The system guarantees that if multiple processes perform operations on the same shared object nearly simultaneously, the net effect is that it looks like the operations took place strictly sequentially, with no operation beginning until the previous one finished.

Furthermore, the operations appear in the same order to all processes. For example, suppose that we were to augment the *stack* object with a new operation, *peek*, to inspect the top item on the stack. Then if two independent processes push 3 and 4 simultaneously, and all processes later use *peek* to examine the top of the stack, the system guarantees that either every process will see 3 or every process will see 4. A situation in which some processes see 3 and other processes see 4 cannot occur in a multiprocessor or a paged-based distributed shared memory, and it cannot occur in Orca either. If only one copy of the stack is maintained, this effect is trivial to achieve, but if the stack is replicated on all machines, more effort is required, as described below.

Although we have not emphasized it, Orca integrates shared data and synchronization in a way not present in page-based DSM systems. Two kinds of synchronization are needed in parallel programs. The first kind is mutual exclusion synchronization, to keep two processes from executing the same critical region at the same time. In Orca, each operation on a shared object is

effectively like a critical region because the system guarantees that the final result is the same as if all the critical regions were executed one at a time (i.e., sequentially). In this respect, an Orca object is like a distributed form of a monitor (Hoare, 1975).

The other kind of synchronization is condition synchronization, in which a process blocks waiting for some condition to hold. In Orca, condition synchronization is done with guards. In the example of Fig. 6-41, a process trying to pop an item from an empty stack will be suspended until the stack is no longer empty.

Management of Shared Objects in Orca

Object management in Orca is handled by the runtime system. It works on both broadcast (or multicast) networks and point-to-point networks. The runtime system handles object replication, migration, consistency, and operation invocation.

Each object can be in one of two states: single copy or replicated. An object in single-copy state exists on only one machine. A replicated object is present on all machines containing a process using it. It is not required that all objects be in the same state, so some of the objects used by a process may be replicated while others are single copy. Objects can change from single-copy state to replicated state and back during execution.

The big advantage of replicating an object on every machine is that reads can be done locally, without any network traffic or delay. When an object is not replicated, all operations must be sent to the object, and the caller must block waiting for the reply. A second advantage of replication is increased parallelism: multiple read operations can take place at the same time. With a single copy, only one operation at a time can take place, slowing down execution. The principal disadvantage of replication is the overhead of keeping all the copies consistent.

When a program performs an operation on an object, the compiler calls a runtime system procedure, *invoke_op*, specifying the object, the operation, the parameters, and a flag telling whether the object will be modified (called a write) or not modified (called a read). The action taken by *invoke_op* depends on whether the object is replicated, whether a copy is available locally, whether it is being read or written, and whether the underlying system supports reliable, totally-ordered broadcasting. Four cases have to be distinguished, as illustrated in Fig. 6-42.

In Fig. 6-42(a), a process wants to perform an operation on a nonreplicated object that happens to be on its own machine. It just locks the object, invokes the operation, and unlocks the object. The point of locking it is to inhibit any remote invocations temporarily while the local operation is in progress.

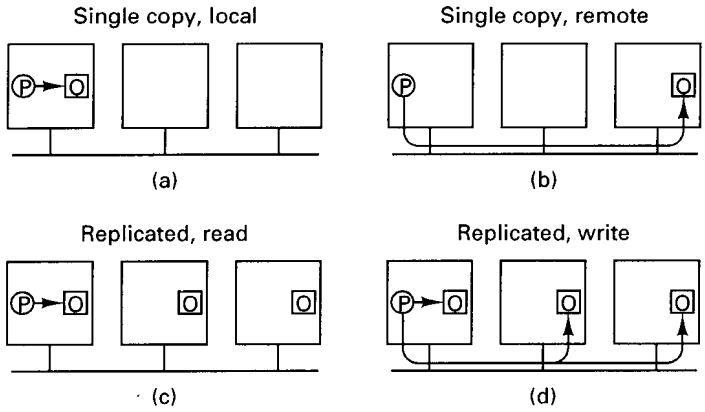


Fig. 6-42. Four cases of performing an operation on an object, *O*.

In Fig. 6-42(b) we still have a single-copy object, but now it is somewhere else. The runtime system does an RPC with the remote machine asking it to perform the operation, which it does, possibly with a slight delay if the object was locked when the request came in. In neither of these two cases is a distinction made between reads and writes (except that writes can awaken blocked processes).

If the object is replicated, as in Fig. 6-42(c) and (d), a copy will always be available locally, but now it matters if the operation is a read or a write. Reads are just done locally, with no network traffic and thus no overhead.

Writes on replicated objects are trickier. If the underlying system provides *reliable*, totally-ordered broadcasting, the runtime system broadcasts the name of the object, the operation, and the parameters and blocks until the broadcast has completed. All the machines, including itself, then compute the new value.

Note that the broadcasting primitive must be reliable, meaning that lower layers automatically detect and recover from lost messages. The Amoeba system, on which Orca was developed, has such a feature. Although the algorithm will be described in detail in Chap. 7, we will summarize it here very briefly. Each message to be broadcast is sent to a special process called the **sequencer**, which assigns it a sequence number and then broadcasts it using the unreliable hardware broadcast. Whenever a process notices a gap in the sequence numbers, it knows that it has missed a message and takes action to recover.

If the system does not have reliable broadcasting (or does not have any broadcasting at all), the update is done using a two-phase, primary copy algorithm. The process doing the update first sends a message to the primary copy of the object, locking and updating it. The primary copy then sends individual messages to all other machines holding the object, asking them to lock their

copies. When all of them have acknowledged setting the lock, the originating process enters the second phase and sends another message telling them to perform the update and unlock the object.

Deadlocks are impossible because even if two processes try to update the same object at the same time, one of them will get to the primary copy first to lock it, and the other request will be queued until the object is free again. Also note that during the update process, all copies of the object are locked, so no other processes can read the old value. This locking guarantees that all operations are executed in a globally unique sequential order.

Notice that this runtime system uses an update algorithm rather than an invalidation algorithm as most page-based DSM systems do. Most objects are relatively small, so sending an update message (the new value or the parameters) is often no more expensive than an invalidate message. Updating has the great advantage of allowing subsequent remote reads to occur without having to refetch the object or perform the operation remotely.

Now let us briefly look at the algorithm for deciding whether an object should be in single-copy state or replicated. Initially, an Orca program consists of one process, which has all the objects. When it forks, all other machines are told of this event and given current copies of all the child's shared parameters. Each runtime system then calculates the expected cost of having each object replicated versus having it not replicated.

To make this calculation, it needs to know the expected ratio of reads to writes. The compiler estimates this information by examining the program, taking into account that accesses inside loops count more and accesses inside if-statements count less than other accesses. Communication costs are also factored into the equation. For example, an object with a read/write ratio of 10 on a broadcast network will be replicated, whereas an object with a read/write ratio of 1 on a point-to-point network will be put in single-copy state, with the single copy going on the machine doing the most writes. For a more detailed description, see (Bal and Kaashoek, 1993).

Since all runtime systems make the same calculation, they come to the same conclusion. If an object currently is present on only one machine and needs to be on all, it is disseminated. If it is currently replicated and that is no longer the best choice, all machines but one discard their copy. Objects can migrate via this mechanism.

Let us see how sequential consistency is achieved. For objects in single-copy state, all operations genuinely are serialized, so sequential consistency is achieved for free. For replicated objects, writes are totally ordered either by the reliable, totally-ordered broadcast or by the primary copy algorithm. Either way, there is global agreement on the order of the writes. The reads are local and can be interleaved with the writes in an arbitrary way without affecting sequential consistency.

Assuming that a reliable, totally-ordered broadcast can be done in two (plus epsilon) messages, as in Amoeba, the Orca scheme is highly efficient. Only after an operation is finished are the results sent out, no matter how many local variables were changed by the operation. If one regards each operation as a critical region, the efficiency is the same as for release consistency—one broadcast at the end of each critical region.

Various optimizations are possible. For example, instead of synchronizing *after* an operation, it could be done when an operation is started, as in entry consistency or lazy release consistency. The advantage here is that if a process executes an operation on a shared object repeatedly (e.g., in a loop), no broadcasts at all are sent until some other process exhibits interest in the object.

Another optimization is not to suspend the caller while doing the broadcast after a write that does not return a value (e.g., *push* in our stack example). Of course, this optimization must be done in such a transparent way. Information supplied by the compiler makes other optimizations possible.

In summary, the Orca model of distributed shared memory integrates good software engineering practice (encapsulated objects), shared data, simple semantics, and synchronization in a natural way. Also, in many cases an implementation as efficient as release consistency is possible. It works best when the underlying hardware and operating system must provide efficient, reliable, totally-ordered broadcasting, and the application must has an inherently high ratio of reads to writes for accesses to shared objects.

6.7. COMPARISON

Let us now briefly compare the various systems we have examined. IVY just tries to mimic a multiprocessor by doing paging over the network instead of to a disk. It offers a familiar memory model—sequential consistency, and can run existing multiprocessor programs without modification. The only problem is the performance.

Munin and Midway try to improve the performance by requiring the programmer to mark those variables that are shared and by using weaker consistency models. Munin is based on release consistency, and on every release transmits all modified pages (as deltas) to other processes sharing those pages. Midway, in contrast, does communication only when a lock changes ownership.

Midway supports only one kind of shared data variable, whereas Munin has four kinds (read only, migratory, write-shared, and conventional). On the other hand, Midway supports three different consistency protocols (entry, release, and processor), whereas Munin only supports release consistency. Whether it is better to have multiple types of shared data or multiple protocols is open to discussion. More research will be needed before we understand this subject fully.

Finally, the way writes to shared variables are detected is different. Munin uses the MMU hardware to trap writes, whereas Midway offers a choice between a special compiler that records writes and doing it the way Munin does, with the MMU. Not having to take a stream of page faults, especially inside critical regions, is definitely an advantage for Midway.

Now let us compare Munin and Midway to object-based shared memory of the Linda-Orca variety. Synchronization and data access in Munin and Midway are up to the programmer, whereas they are tightly integrated in Linda and Orca. In Linda there is less danger that a programmer will make a synchronization error, since *in* and *out* handle their own synchronization internally. Similarly, when an operation on a shared object is invoked in Orca, the locking is handled completely by the runtime system, with the programmer not even being aware of it. Condition synchronization (as opposed to mutual exclusion synchronization) is not part of the Munin or Midway model, so it is up to the programmer to do all the work explicitly. In contrast, it is an integral part of the Linda model (blocking when a tuple is not present) and the Orca model (blocking on a guard).

In short, the Munin and Midway programmers must do more work in the area of synchronization and consistency, with little support, and must get it all right. There is no encapsulation and there are no methods to protect shared data, as Linda and Orca have. On the other hand, Munin and Midway allow programming in only slightly modified C or C++, whereas Linda's communication is unusual and Orca is a whole new language. In terms of efficiency, Midway is best in terms of the number and size of messages transmitted, although the use of fundamentally different programming models (open C code, objects, and tuples) may lead to substantially different algorithms in the three cases, which also can affect efficiency.

6.8. SUMMARY

Multiple CPU computer systems fall into two categories: those that have shared memory and those that do not. The shared memory machines (multiprocessors) are easier to program but harder to build, whereas the machines without shared memory (multicomputers) are harder to program but easier to build. Distributed shared memory is a technique for making multicomputers easier to program by simulating shared memory on them.

Small multiprocessors are often bus based, but large ones are switched. The protocols the large ones use require complex data structures and algorithms to keep the caches consistent. NUMA multiprocessors avoid this complexity by forcing the software to make all the decisions about which pages to place on which machine.

A straightforward implementation of DSM, as done in IVY, is possible, but

often has substantially lower performance than a multiprocessor. For this reason, researchers have looked at various memory models in an attempt to obtain better performance. The standard against which all models are measured is sequential consistency, which means that all processes see all memory references in the same order. Causal consistency, PRAM consistency, and processor consistency all weaken the concept that processes see *all* memory references in the same order.

Another approach is that of weak consistency, release consistency, and entry consistency, in which memory is not consistent all the time, but the programmer can force it to become consistent by certain actions, such as entering or leaving a critical region.

Three general techniques have been used to provide DSM. The first simulates the multiprocessor memory model, giving each process a linear, paged memory. Pages are moved back and forth between machines as needed. The second uses ordinary programming languages with annotated shared variables. The third is based on higher-level programming models such as tuples and objects.

In this chapter we studied five different approaches to DSM. IVY operates essentially like a virtual memory, with pages moved from machine to machine when page faults occur. Munin uses multiple protocols and release consistency to allow individual variables to be shared. Midway is similar to Munin, except that it uses entry consistency instead of release consistency as the normal case. Linda represents the other end of the spectrum, with an abstract tuple space far removed from the details of paging. Orca supports a model in which data objects are replicated on multiple machines and accessed through protected methods that make the objects look sequentially consistent to the programmer.

PROBLEMS

1. A Dash system has b bytes of memory divided over n clusters. Each cluster has p processors in it. The cache block size is c bytes. Give a formula for the total amount of memory devoted to directories (excluding the two state bits per directory entry).
2. Is it really essential that Dash make a distinction between the CLEAN and UNCACHED states? Would it have been possible to dispense with one of them? After all, in both cases memory has an up-to-date copy of the block.
3. In the text it is stated that many minor variations of the cache ownership protocol of Fig. 6-6 are possible. Describe one such variation and give one advantage yours has over the one in the text.

4. Why is the concept of "home memory" needed in Memnet but not in Dash?

5. In a NUMA multiprocessor, local memory references take 100 nsec and remote references take 800 nsec. A certain program makes a total of N memory references during its execution, of which 1 percent are to a page P. That page is initially remote, and it takes C nsec to copy it locally. Under what conditions should the page be copied locally in the absence of significant use by other processors?

6. During the discussion of memory consistency models, we often referred to the contract between the software and memory. Why is such a contract needed?

7. A multiprocessor has a single bus. Is it possible to implement strictly consistent memory?

8. In Fig. 6-13(a) an example of a sequentially consistent memory is shown. Make a minimal change to P_2 that violates sequential consistency.

9. In Fig. 6-14, is 001110 a legal output for a sequentially consistent memory? Explain your answer.

10. At the end of Sec. 6-2.2, we discussed a formal model that said every set of operations on a sequentially consistent memory can be modeled by a string, S, from which all the individual process sequences can be derived. For processes P_1 and P_2 in Fig. 6-16, give all the possible values of S. Ignore processes P_3 and P_4 and do not include their memory references in S.

11. In Fig. 6-20, a sequentially consistent memory allows six possible statement interleavings. List them all.

12. Why is $W(x)1$ $R(x)0$ $R(x)1$ not legal for Fig. 6-12(b)?

13. In Fig. 6-14, is 000000 a legal output for a memory that is only PRAM consistent? Explain your answer.

14. In most implementations of (eager) release consistency in DSM systems, shared variables are synchronized on a release, but not on an acquire. Why is acquire needed at all then?

15. In Fig. 6-27, suppose that the page owner is located by broadcasting. In which of the cases should the page be sent for a read?

16. In Fig. 6-28 a process may contact the owner of a page via the page manager. It may want ownership or the page itself, which are independent quantities. Do all four cases exist (excepting, of course, the case where the requester does not want the page and does not want ownership)?

17. Suppose that two variables, *A* and *B* are both located, by accident, on the same page of a page-based DSM system. However, both of them are unshared variables. Is false sharing possible?

18. Why does IVY use an invalidation scheme for consistency instead of an update scheme?

19. Some machines have a single instruction that, in one atomic action, exchanges a register with a word in memory. Using this instruction, it is possible to implement semaphores for protecting critical regions. Will programs using this instruction work on a page-based DSM system? If so, under what circumstances will they work efficiently?

20. What happens if a Munin process modifies a shared variable outside a critical region?

21. Give an example of an *in* operation in Linda that does not require any searching or hashing to find a match.

22. When Linda is implemented by replicating tuples on multiple machines, a protocol is needed for deleting tuples. Give an example of a protocol that does not yield races when two processes try to delete the same tuple at the same time.

23. Consider an Orca system running on a network with hardware broadcasting. Why does the ratio of read operations to write operations affect the performance?

7

Case Study 1: Amoeba

In this chapter we will give our first example of a distributed operating system: Amoeba. In the following one we will look at a second example: Mach. Amoeba is a distributed operating system: it makes a collection of CPUs and I/O equipment act like a single computer. It also provides facilities for parallel programming where that is desired. This chapter describes the goals, design, and implementation of Amoeba. For more information about Amoeba, see (Mullender et al., 1990; and Tanenbaum et al., 1990).

7.1. INTRODUCTION TO AMOEBA

In this section we will give an introduction to Amoeba, starting with a brief history and its current research goals. Then we will look at the architecture of a typical Amoeba system. Finally, we will begin our study of the Amoeba software, both the kernel and the servers.

7.1.1. History of Amoeba

Amoeba originated at the Vrije Universiteit, Amsterdam, The Netherlands in 1981 as a research project in distributed and parallel computing. It was designed primarily by Andrew S. Tanenbaum and three of his Ph.D. students, Frans

Kaashoek, Sape J. Mullender, and Robbert van Renesse, although many other people also contributed to the design and implementation. By 1983, an initial prototype, Amoeba 1.0, was operational.

Starting in 1984, the Amoeba fissioned, and a second group was set up at the Centre for Mathematics and Computer Science, also in Amsterdam, under Mullender's leadership. In the succeeding years, this cooperation was extended to sites in England and Norway in a wide-area distributed system project sponsored by the European Community. This work used Amoeba 3.0, which unlike the earlier versions, was based on RPC. Using Amoeba 3.0, it was possible for clients in Tromso to access servers in Amsterdam transparently, and vice versa.

The system evolved for several years, acquiring such features as partial UNIX emulation, group communications and a new low-level protocol. The version described in this chapter is Amoeba 5.2.

7.1.2. Research Goals

Many research projects in distributed operating systems have started with an existing system (e.g., UNIX) and added new features, such as networking and a shared file system, to make it more distributed. The Amoeba project took a different approach. It started with a clean slate and developed a new system from scratch. The idea was to make a fresh start and experiment with new ideas without having to worry about backward compatibility with any existing system. To avoid the chore of having to rewrite a huge amount of application software from scratch as well, a UNIX emulation package was added later.

The primary goal of the project was to build a transparent distributed operating system. To the average user, using Amoeba is like using a traditional timesharing system like UNIX. One logs in, edits and compiles programs, moves files around, and so on. The difference is that each of these actions makes use of multiple machines over the network. These include process servers, file servers, directory servers, compute servers, and other machines, but the user is not aware of any of this. At the terminal, it just looks like a timesharing system.

An important distinction between Amoeba and most other distributed systems is that Amoeba has no concept of a "home machine." When a user logs in, it is to the system as a whole, not to a specific machine. Machines do not have owners. The initial shell, started upon login, runs on some arbitrary machine, but as commands are started up, in general they do not run on the same machine as the shell. Instead, the system automatically looks around for the most lightly loaded machine to run each new command on. During the course of a long terminal session, the processes that run on behalf of any one user will be spread more-or-less uniformly spread over all the machines in the system, depending on the load, of course. In this respect, Amoeba is higly location transparent.

In other words, all resources belong to the system as a whole and are

managed by it. They are not dedicated to specific users, except for short periods of time to run individual processes. This model attempts to provide the transparency that is the holy grail of all distributed systems designers.

A simple example is *amake*, the Amoeba replacement for the UNIX *make* program. When the user types *amake*, all the necessary compilations happen, as expected, except that the system (and not the user) determines whether they happen sequentially or in parallel, and on which machine or machines this occurs. None of this is visible to the user.

A secondary goal of Amoeba is to provide a testbed for doing distributed and parallel programming. While some users just use Amoeba the same way they would use any other timesharing system most users are specifically interested in experimenting with distributed and parallel algorithms, languages, tools, and applications. Amoeba supports these users by making the underlying parallelism available to people who want to take advantage of it. In practice, most of Amoeba's current user base consists of people who are specifically interested in distributed and parallel computing in its various forms. A language, Orca, has been specifically designed and implemented on Amoeba for this purpose. Orca and its applications are described in (Bal, 1991; Bal et al., 1990; and Tanenbaum et al., 1992). Amoeba itself, however, is written in C.

7.1.3. The Amoeba System Architecture

Before describing how Amoeba is structured, it is useful first to outline the kind of hardware configuration for which Amoeba was designed, since it differs somewhat from what most organizations presently have. Amoeba was designed with two assumptions about the hardware in mind:

1. Systems will have a very large number of CPUs.
2. Each CPU will have tens of megabytes of memory.

These assumptions are already true at some installations, and will probably become true at almost all corporate, academic, and governmental sites within a few years.

The driving force behind the system architecture is the need to incorporate large numbers of CPUs in a straightforward way. In other words, what do you do when you can afford 10 or 100 CPUs per user? One solution is to give each user a personal 10-node or 100-node multiprocessor.

Although giving everyone a personal multiprocessor is certainly a possibility, doing so is not an effective way to spend the available budget. Most of the time, nearly all the processors will be idle, but some users will want to run massively parallel programs and will not be able to harness all the idle CPU cycles because they are in other users' personal machines.

Instead of this personal multiprocessor approach, Amoeba is based on the model shown in Fig. 7-1. In this model, all the computing power is located in one or more **processor pools**. A processor pool consists of a substantial number of CPUs, each with its own local memory and network connection. Shared memory is not required, or even expected, but if it is present it could be used to optimize message passing by doing memory-to-memory copying instead of sending messages over the network.

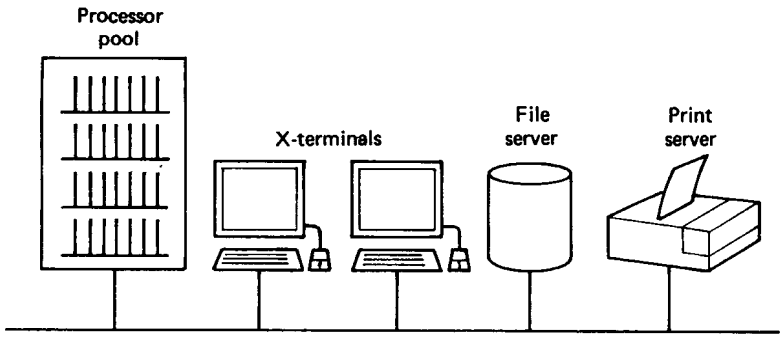


Fig. 7-1. The Amoeba system architecture.

The CPUs in a pool can be of different architectures, for example, a mixture of 680x0, 386, and SPARC machines. Amoeba has been designed to deal with multiple architectures and heterogeneous systems. It is even possible for the children of a single process to run on different architectures.

Pool processors are not "owned" by any one user. When a user types a command, the operating system dynamically chooses one or more processors on which to run that command. When the command completes, the processes are terminated and the resources held go back into the pool, waiting for the next command, very likely from a different user. If there is a shortage of pool processors, individual processors are timeshared, with new processes being assigned to the most lightly loaded CPUs. The important point to note here is that this model is quite different from current systems in which each user has exactly one personal workstation for all his computing activities.

The expected presence of large memories in future systems has influenced the design in many ways. Many time-space trade-offs have been made to provide high performance at the cost of using more memory. We will see examples later.

The second element of the Amoeba architecture is the terminal. It is through the terminal that the user accesses the system. A typical Amoeba terminal is an X terminal, with a large bit-mapped screen and a mouse. Alternatively, a personal computer or workstation running X windows can also be used as a terminal. Although Amoeba does not forbid running user programs on the

terminal, the idea behind this model is to give the users relatively cheap terminals and concentrate the computing cycles into a common pool so that they can be used more efficiently.

Pool processors are inherently cheaper than workstations because they consist of just a single board with a network connection. There is no keyboard, monitor, or mouse, and the power supply can be shared by many boards. Thus, instead of buying 100 high-performance workstations for 100 users, one might buy 50 high-performance pool processors and 100 X terminals for the same price (depending on the economics, obviously). Since the pool processors are allocated only when needed, an idle user only ties up an inexpensive X terminal instead of an expensive workstation. The trade-offs inherent in the pool processor model versus the workstation model were discussed in Chap. 4.

To avoid any confusion, the pool processors do not *have* to be single-board computers. If these are not available, a subset of the existing personal computers or workstations can be designated as pool processors. They also do not need to be located in one room. The physical location is actually irrelevant. The pool processors can even be in different countries, as we will discuss later.

Another important component of the Amoeba configuration consists of specialized servers, such as file servers, which for hardware or software reasons need to run on a separate processor. In some cases a server is able to run on a pool processor, being started up as needed, but for performance reasons it is better to have it running all the time.

Servers provide services. A **service** is an abstract definition of what the server is prepared to do for its clients. This definition defines what the client can ask for and what the results will be, but it does not specify how many servers are working together to provide the service. In this way, the system has a mechanism for providing fault-tolerant services by having multiple servers doing the work.

An example is the directory server. There is nothing inherent about the directory server or the system design that would prevent a user from starting up a new directory server on a pool processor every time he wanted to look up a file name. However, doing so would be horrendously inefficient, so one or more directory servers are kept running all the time, generally on dedicated machines to enhance their performance. The decision to have some servers always running and others to be started explicitly when needed is up to the system administrator.

7.1.4. The Amoeba Microkernel

Having looked at the Amoeba hardware model, let us now turn to the software model. Amoeba consists of two basic pieces: a microkernel, which runs on every processor, and a collection of servers that provide most of the

traditional operating system functionality. The overall structure is shown in Fig. 7-2.

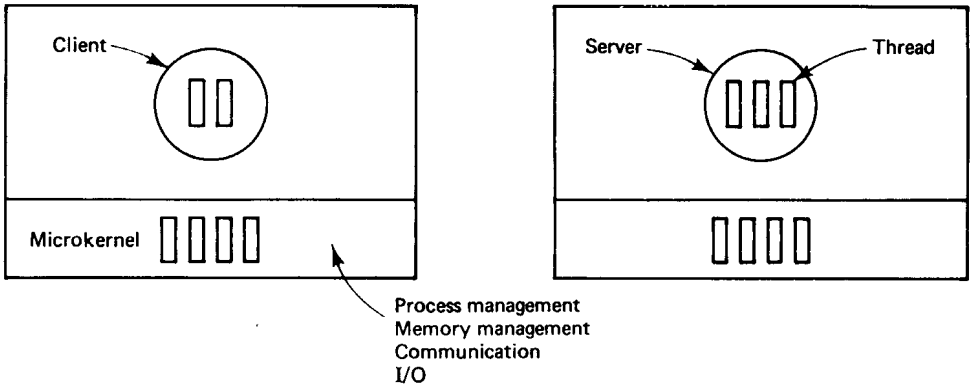


Fig. 7-2. Amoeba software structure.

The Amoeba microkernel runs on all machines in the system. The same kernel can be used on the pool processors, the terminals (assuming that they are computers, rather than X terminals), and the specialized servers. The microkernel has four primary functions:

1. Manage processes and threads.
2. Provide low-level memory management support.
3. Support communication.
4. Handle low-level I/O.

Let us consider each of these in turn.

Like most operating systems, Amoeba supports the concept of a process. In addition, Amoeba also supports multiple threads of control within a single address space. A process with one thread is essentially the same as a process in UNIX. Such a process has a single address space, a set of registers, a program counter, and a stack.

In contrast, although a process with multiple threads still has a single address space shared by all threads, each thread logically has its own registers, its own program counter, and its own stack. In effect, a collection of threads in a process is similar to a collection of independent processes in UNIX, with the one exception that they all share a single common address space.

A typical use for multiple threads might be in a file server, in which every incoming request is assigned to a separate thread to work on. That thread might begin processing the request, then block waiting for the disk, then continue

work. By splitting the server up into multiple threads, each thread can be purely sequential, even if it has to block waiting for I/O. Nevertheless, all the threads can, for example, have access to a single shared software cache. Threads can synchronize using semaphores or mutexes to prevent two threads from accessing the shared cache simultaneously.

The second task of the kernel is to provide low-level memory management. Threads can allocate and deallocate blocks of memory, called **segments**. These segments can be read and written, and can be mapped into and out of the address space of the process to which the calling thread belongs. A process must have at least one segment, but it may also have many more of them. Segments can be used for text, data, stack, or any other purpose the process desires. The operating system does not enforce any particular pattern on segment usage.

The third job of the kernel is to handle interprocess communication. Two forms of communication are provided: point-to-point communication and group communication. These are closely integrated to make them similar.

Point-to-point communication is based on the model of a client sending a message to a server, then blocking until the server has sent a reply back. This request/reply exchange is the basis on which almost everything else is built.

The other form of communication is group communication. It allows a message to be sent from one source to multiple destinations. Software protocols provide reliable, fault-tolerant group communication to user processes in the presence of lost messages and other errors.

The fourth function of the kernel is to manage low-level I/O. For each I/O device attached to a machine, there is a device driver in the kernel. The driver manages all I/O for the device. Drivers are linked with the kernel and cannot be loaded dynamically.

Device drivers communicate with the rest of the system by the standard request and reply messages. A process, such as a file server, that needs to communicate with the disk driver, sends it request messages and gets back replies. In general, the client does not have to know that it is talking to a driver. As far as it is concerned, it is just communicating with a thread somewhere.

Both the point-to-point message system and the group communication make use of a specialized protocol called FLIP. This protocol is a network layer protocol and has been designed specifically to meet the needs of distributed computing. It deals with both unicasting and multicasting on complex internetworks. It will be discussed later.

7.1.5. The Amoeba Servers

Everything that is not done by the kernel is done by server processes. The idea behind this design is to minimize kernel size and enhance flexibility. By not building the file system and other standard services into the kernel, they can

be changed easily, and multiple versions can run simultaneously for different user populations.

Amoeba is based on the client-server model. Clients are typically written by the users and servers are typically written by the system programmers, but users are free to write their own servers if they wish. Central to the entire software design is the concept of an object, which is like an abstract data type. Each object consists of some encapsulated data with certain operations defined on it. File objects have a READ operation, for example, among others.

Objects are managed by servers. When a process creates an object, the server that manages the object returns to the client a cryptographically protected capability for the object. To use the object later, the proper capability must be presented. All objects in the system, both hardware and software, are named, protected, and managed by capabilities. Among the objects supported this way are files, directories, memory segments, screen windows, processors, disks, and tape drives. This uniform interface to all objects provides generality and simplicity.

All the standard servers have stub procedures in the library. To use a server, a client normally just calls the stub, which marshals the parameters, sends the message, and blocks until the reply comes back. This mechanism hides all the details of the implementation from the user. A stub compiler is available for users who wish to produce stub procedures for their own servers.

Probably the most important server is the file server, known as the **bullet server**. It provides primitives to manage files, creating them, reading them, deleting them, and so on. Unlike most file servers, the files it creates are immutable. Once created, a file cannot be modified, but it can be deleted. Immutable files make automatic replication easier since they avoid many of the race conditions inherent in replicating files that are subject to being changed during the replication process.

Another important server is the **directory server**, for obscure historical reasons also known as the **soap server**. It is the directory server that manages directories and path names and maps them onto capabilities. To read a file, a process asks the directory server to look up the path name. On a successful lookup, the directory server returns the capability for the file (or other object). Subsequent operations on the file do not use the directory server, but go straight to the file server. Splitting the file system into these two components increases flexibility and makes each one simpler, since it only has to manage one type of object (directories or files), not two.

Other standard servers are present for handling object replication, starting processes, monitoring servers for failures, and communicating with the outside world. User servers perform a wide variety of application-specific tasks.

The rest of this chapter is structured as follows. First we will describe objects and capabilities, since these are the heart of the entire system. Then we

will look at the kernel, focusing on process management, memory management, and communication. Finally, we will examine some of the main servers, including the bullet server, the directory server, the replication server, and the run server.

7.2. OBJECTS AND CAPABILITIES IN AMOEBA

The basic unifying concept underlying all the Amoeba servers and the services they provide is the **object**. An object is an encapsulated piece of data upon which certain well-defined operations may be performed. It is, in essence, an abstract data type. Objects are passive. They do not contain processes or methods or other active entities that "do" things. Instead, each object is managed by a server process.

To perform an operation on an object, a client does an RPC with the server, specifying the object, the operation to be performed, and optionally, any parameters needed. The server does the work and returns the answer. Operations are performed synchronously, that is, after initiating an RPC with a server to get some work done, the client thread is blocked until the server replies. Other threads in the same process are still runnable, however.

Clients are unaware of the locations of the objects they use and the servers that manage these objects. A server might be running on the same machine as the client, on a different machine on the same LAN, or even on a machine thousands of kilometers away. Furthermore, although most servers run as user processes, a few low-level ones, such as the segment (i.e., memory) server and process server, run as threads in the kernel. This distinction, too, is invisible to clients. The RPC protocol for talking to user servers or kernel servers, whether local or remote, is identical in all cases. Thus a client is concerned entirely with what it wants to do, not where objects are stored and where servers run. A certain directory contains the capabilities for all the accessible file servers along with a specification of the default choice, so a user can override the default in cases where it matters. Usually, the system administrator sets up the default to be the local one.

7.2.1. Capabilities

Objects are named and protected in a uniform way, by special tickets called **capabilities**. To create an object, a client does an RPC with the appropriate server specifying what it wants. The server then creates the object and returns a capability to the client. On subsequent operations, the client must present the capability to identify the object. A capability is just a long binary number. The Amoeba 5.2 format is shown in Fig. 7-3.

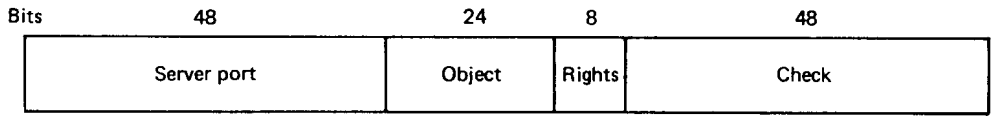


Fig. 7-3. A capability in Amoeba.

When a client wants to perform an operation on an object, it calls a stub procedure that builds a message containing the object's capability and then traps to the kernel. The kernel extracts the *Server port* field from the capability and looks it up in its cache to locate the machine on which the server resides. If the port is not in the cache, it is located by broadcasting, as will be described later. The port is effectively a logical address at which the server can be reached. Server ports are thus associated with a particular server (or a set of servers), not with a specific machine. If a server moves to a new machine, it takes its server port with it. Many server ports, like that of the file server, are publicly known and stable for years. The only way a server can be addressed is via its port, which it initially chose itself.

The rest of the information in the capability is ignored by the kernels and passed to the server for its own use. The *Object* field is used by the server to identify the specific object in question. For example, a file server might manage thousands of files, with the object number being used to tell it which one is being operated on. In a sense, the *Object* field in a file capability is analogous to a UNIX i-node number.

The *Rights* field is a bit map telling which of the allowed operations the holder of a capability may perform. For example, although a particular object may support reading and writing, a specific capability may be constructed with all the rights bits except READ turned off.

The *Check* field is used for validating the capability. Capabilities are manipulated directly by user processes. Without some form of protection, there would be no way to prevent user processes from forging capabilities.

7.2.2. Object Protection

The basic algorithm used to protect objects is as follows. When an object is created, the server picks a random *Check* field and stores it both in the new capability and inside its own tables. All the rights bits in a new capability are initially on, and it is this **owner capability** that is returned to the client. When the capability is sent back to the server in a request to perform an operation, the *Check* field is verified.

To create a restricted capability, a client can pass a capability back to the server, along with a bit mask for the new rights. The server takes the original

Check field from its tables, EXCLUSIVE ORs it with the new rights (which must be a subset of the rights in the capability), and then runs the result through a one-way function. Such a function, $y = f(x)$, has the property that given x it is easy to find y, but given only y, finding x requires an exhaustive search of all possible x values (Evans et al., 1974).

The server then creates a new capability, with the same value in the *Object* field, but the new rights bits in the *Rights* field and the output of the one-way function in the *Check* field. The new capability is then returned to the caller. The client may send this new capability to another process, if it wishes, as capabilities are managed entirely in user space.

The method of generating restricted capabilities is illustrated in Fig. 7-4. In this example, the owner has turned off all the rights except one. For example, the restricted capability might allow the object to be read, but nothing else. The meaning of the *Rights* field is different for each object type since the legal operations themselves also vary from object type to object type.

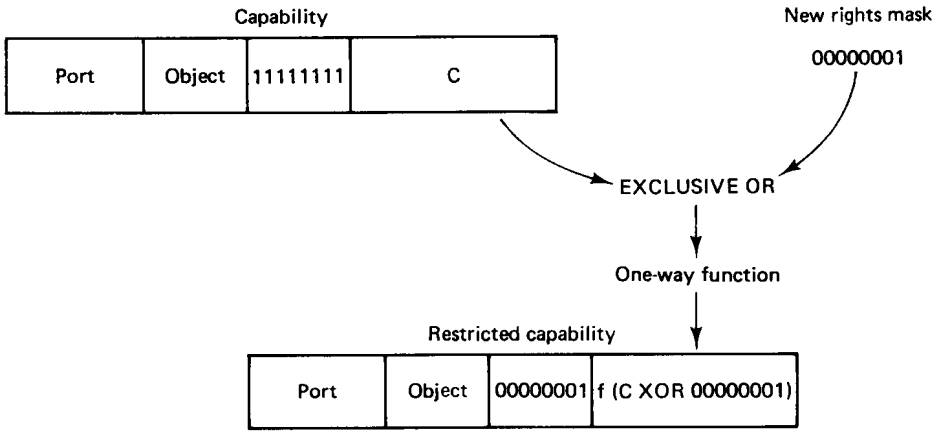


Fig. 7-4. Generation of a restricted capability from an owner capability.

When the restricted capability comes back to the server, the server sees from the *Rights* field that it is not an *owner capability* because at least one bit is turned off. The server then fetches the original random number from its tables, EXCLUSIVE ORs it with the *Rights* field from the capability, and runs the result through the one-way function. If the result agrees with the *Check* field, the capability is accepted as valid.

It should be obvious from this algorithm that a user who tries to add rights that he does not have will simply invalidate the capability. Inverting the *Check* field in a restricted capability to get the argument (*C* XOR 00000001 in Fig. 7-4) is impossible because the function f is a one-way function (that is what "one-

way" means—no algorithm exists for inverting it). It is through this cryptographic technique that capabilities are protected from tampering.

Capabilities are used throughout Amoeba for both naming of all objects and for protecting them. This single mechanism leads to a uniform naming and protection scheme. It also is fully location transparent. To perform an operation on an object, it is not necessary to know where the object resides. In fact, even if this knowledge were available, there would be no way to use it.

Note that Amoeba does not use access control lists for authentication. The protection scheme used requires almost no administrative overhead. However, in an insecure environment, additional cryptography (e.g., link encryption) may be required to keep capabilities from being disclosed accidentally to wiretappers on the network.

7.2.3. Standard Operations

Although many operations on objects depend on the object type, there are some operations that apply to most objects. These are listed in Fig. 7-5. Some of these require certain rights bits to be set, but others can be done by anyone who can present a server with a valid capability for one of its objects.

Call	Description
Age	Perform a garbage collection cycle
Copy	Duplicate the object and return a capability for the copy
Destroy	Destroy the object and reclaim its storage
Getparams	Get parameters associated with the server
Info	Get an ASCII string briefly describing the object
Restrict	Produce a new, restricted capability for the object
Setparams	Set parameters associated with the server
Status	Get current status information from the server
Touch	Pretend the object was just used

Fig. 7-5. The standard operations valid on most objects.

It is possible to create an object in Amoeba and then lose the capability, so some mechanism is needed to get rid of old objects that are no longer accessible. The way that has been chosen is to have servers run a garbage collector

periodically, removing all objects that have not been used in n garbage collection cycles. The AGE call starts a new garbage collection cycle. The TOUCH call tells the server that the object touched is still in use. When objects are entered into the directory server, they are touched periodically, to keep the garbage collector at bay. Rights for some of the standard operations, such as AGE, are normally present only in capabilities owned by the system administrator.

The COPY operation is a shortcut that makes it possible to duplicate an object without actually transferring it. Without this operation, copying a file would require sending it over the network twice: from the server to the client and then back again. COPY can also fetch remote objects or send objects to remote machines.

The DESTROY operation deletes the object. It always needs the appropriate right, for obvious reasons.

The GETPARAMS and SETPARAMS calls normally deal with the server as a whole rather than with a particular object. They allow the system administrator to read and write parameters that control server operation. For example, the algorithm used to choose processors can be selected using this mechanism.

The INFO and STATUS calls return status information. The former returns a short ASCII string describing the object briefly. The information in the string is server dependent, but in general, it indicates the type of object and tells something useful about it (e.g., for files, it tells the size). The latter gets information about the server as a whole, for example, how much free memory it has. This information helps the system administrator monitor the system better.

The RESTRICT call generates a new capability for the object, with a subset of the current rights, as described above.

7.3. PROCESS MANAGEMENT IN AMOEBA

A process in Amoeba is basically an address space and a collection of threads that run in it. A process with one thread is roughly analogous to a UNIX or MS-DOS process in terms of how it behaves and what it can do. In this section we will explain how processes and threads work, and how they are implemented.

7.3.1. Processes

A process is an object in Amoeba. When a process is created, the parent process is given a capability for the child process, just as with any other newly created object. Using this capability, the child can be suspended, restarted, signaled, or destroyed.

Process creation in Amoeba is different from UNIX. The UNIX model of

creating a child process by cloning the parent is inappropriate in a distributed system due to the considerable overhead of first creating a copy somewhere (FORK) and almost immediately afterward replacing the copy with a new program (EXEC). Instead, in Amoeba it is possible to create a new process on a specific processor with the intended memory image starting right at the beginning. In this one respect, process creation in Amoeba is similar to MS-DOS. However, in contrast to MS-DOS, a process can continue executing in parallel with its child, and thus can create an arbitrary number of additional children. The children can create their own children, leading to a tree of processes.

Process management is handled at three different levels in Amoeba. At the lowest level are the process servers, which are kernel threads running on every machine. To create a process on a given machine, another process does an RPC with that machine's process server, providing it with the necessary information.

At the next level up we have a set of library procedures that provide a more convenient interface for user programs. Several flavors are provided. They do their job by calling the low-level interface procedures.

Finally, the simplest way to create a process is to use the run server, which does most of the work of determining where to run the new process. We will discuss the run server later in this chapter.

Some of the process management calls use a data structure called a **process descriptor** to provide information about the process to be run. One field in the process descriptor (see Fig. 7-6) tells which CPU architecture the process can run on. In heterogeneous systems, this field is essential to make sure that 386 binaries are not run on SPARCs, and so on.

Another field contains the process' owner's capability. When the process terminates or is stunned (see below), RPCs will be done using this capability to report the event. It also contains descriptors for all the process' segments, which collectively define its address space, as well as descriptors for all its threads.

Finally, the process descriptor also contains a descriptor for each thread in the process. The content of a thread descriptor is architecture dependent, but as a bare minimum, it contains the thread's program counter and stack pointer. It may also contain additional information necessary to run the thread, including other registers, the thread's state, and various flags. Brand new processes contain only one thread in their process descriptors, but stunned processes may have created additional threads before being stunned.

The low-level process interface consists of about a half-dozen library procedures. Only three of these will concern us here. The first, *exec*, is the most important. It has two input parameters, the capability for a process server and a process descriptor. Its function is to do an RPC with the specified process server asking it to run the process. If the call is successful, a capability for the new process is returned to the caller for use in controlling the process later.

A second important library procedure is *getload*. It returns information

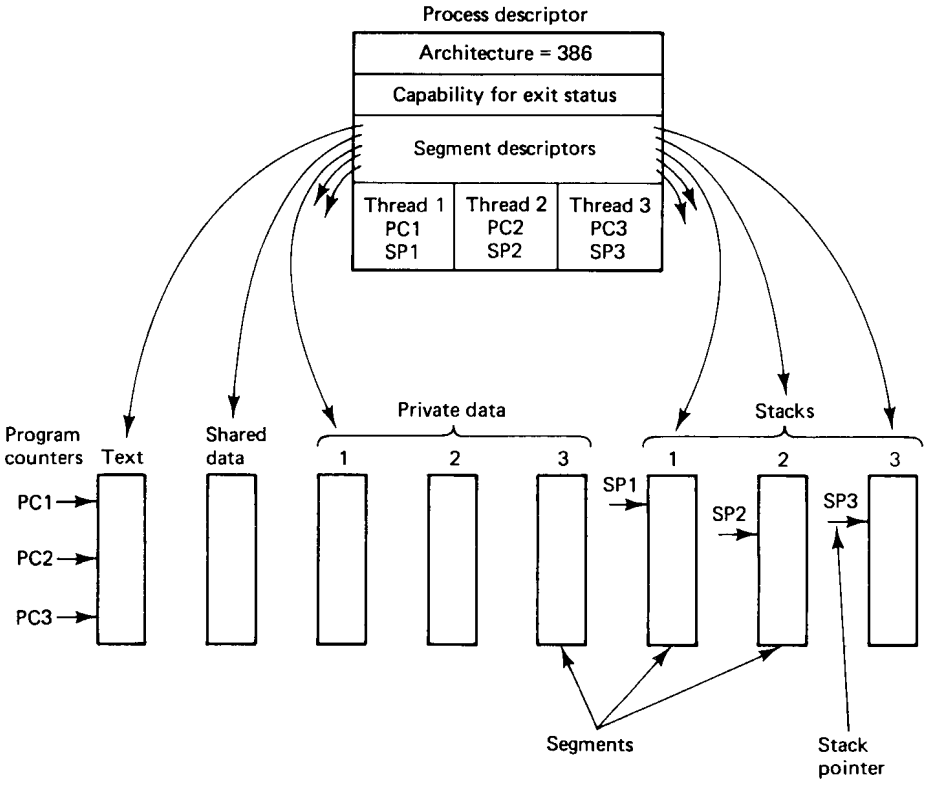


Fig. 7-6. A process descriptor.

about the CPU speed, current load, and amount of memory free at the moment. It is used by the run server to determine the best place to execute a new process.

A third major library procedure is *stun*. A process' parent can suspend it by **stunning** it. More commonly, the parent can give the process' capability to a debugger, which can stun it and later restart it for interactive debugging purposes. Two kinds of stuns are supported: normal and emergency. They differ with respect to what happens if the process is blocked on one or more RPCs at the time it is stunned. With a normal stun, the process sends a message to the server it is currently waiting for, saying, in effect: "I have been stunned. Finish your work instantly and send me a reply." If the server is also blocked, waiting for another server, the message is propagated further, all the way down the line to the end. The server at the end of the line is expected to reply immediately with a special error message. In this way, all the pending RPCs are terminated almost immediately in a clean way, with all of the servers finishing properly. The nesting structure is not violated, and no "long jumps" are needed.

An emergency stun stops the process instantly and does not send any messages to servers that are currently working for the stunned process. The computations being done by the servers become orphans. When the servers finally finish and send replies, these replies are ultimately discarded.

The high-level process interface does not require a fully formed process descriptor. One of the calls, *newproc*, takes as its first three parameters, the name of the binary file and pointers to the argument and environment arrays, similar to UNIX. Additional parameters provide more detailed control of the initial state.

7.3.2. Threads

Amoeba supports a simple threads model. When a process starts up, it has one thread. During execution, the process can create additional threads, and existing threads can terminate. The number of threads is therefore completely dynamic. When a new thread is created, the parameters to the call specify the procedure to run and the size of the initial stack.

Although all threads in a process share the same program text and global data, each thread has its own stack, its own stack pointer, and its own copy of the machine registers. In addition, if a thread wants to create and use variables that are global to all its procedures but invisible to other threads, library procedures are provided for that purpose. Such variables are called **glocal**. One library procedure allocates a block of glocal memory of whatever size is requested, and returns a pointer to it. Blocks of glocal memory are referred to by integers. A system call is available for a thread to acquire its glocal pointer.

Three methods are provided for threads to synchronize: signals, mutexes, and semaphores. Signals are asynchronous interrupts sent from one thread to another thread in the same process. They are conceptually similar to UNIX signals, except that they are between threads rather than between processes. Signals can be raised, caught, or ignored. Asynchronous interrupts between processes use the stun mechanism.

The second form of interthread communication is the mutex. A **mutex** is like a binary semaphore. It can be in one of two states, locked or unlocked. Trying to lock an unlocked mutex causes it to become locked. The calling thread continues. Trying to lock a mutex that is already locked causes the calling thread to block until another thread unlocks the mutex. If more than one thread is waiting on a mutex, when the mutex is unlocked, exactly one thread is released. In addition to the calls to lock and unlock mutexes, there is also one that tries to lock a mutex, but if it is unable to do so within a specified interval, times out and returns an error code. Mutexes are fair and respect thread priorities.

The third way that threads can communicate is by counting semaphores.

These are slower than mutexes, but there are times when they are needed. They work in the usual way, except that here too an additional call is provided to allow a DOWN operation to time out if it is unable to succeed within a specified interval.

All threads are managed by the kernel. The advantage of this design is that when a thread does an RPC, the kernel can block that thread and schedule another one in the same process if one is ready. Thread scheduling is done using priorities, with kernel threads getting higher priority than user threads. Thread scheduling can be set up to be either pre-emptive or run-to-completion (i.e., threads continue to run until they block), as the process wishes.

7.4. MEMORY MANAGEMENT IN AMOEBA

Amoeba has an extremely simple memory model. A process can have any number of segments it wants to have, and they can be located wherever it wants in the process' virtual address space. Segments are not swapped or paged, so a process must be entirely memory resident to run. Furthermore, although the hardware MMU is used, each segment is stored contiguously in memory.

Although this design is perhaps somewhat unusual these days, it was done for three reasons: performance, simplicity, and economics. Having a process entirely in memory all the time makes RPC go faster. When a large block of data must be sent, the system knows that all of the data are contiguous not only in virtual memory, but also in physical memory. This knowledge saves having to check if all the pages containing the buffer happen to be around at the moment, and eliminates having to wait for them if they are not. Similarly, on input, the buffer is always in memory, so the incoming data can be placed there simply and without page faults. This design has allowed Amoeba to achieve extremely high transfer rates for large RPCs.

The second reason for the design is simplicity. Not having paging or swapping makes the system considerably simpler and makes the kernel smaller and more manageable. However, it is the third reason that makes the first two feasible. Memory is becoming so cheap that within a few years, all Amoeba machines will probably have tens of megabytes of it. Such large memories will substantially reduce the need for paging and swapping, namely, to fit large programs into small machines.

7.4.1. Segments

Processes have several calls available to them for managing segments. Most important among these is the ability to create, destroy, read, and write segments. When a segment is created, the caller gets back a capability for it. This

capability is used for reading and writing the segment and for all the other calls involving the segment.

When a segment is created it is given an initial size. This size may change during process execution. The segment may also be given an initial value, either from another segment or from a file.

Because segments can be read and written, it is possible to use them to construct a main memory file server. To start, the server creates a segment as large as it can. It can determine the maximum size by asking the kernel. This segment will be used as a simulated disk. The server then formats the segment as a file system, putting in whatever data structures it needs to keep track of files. After that, it is open for business, accepting requests from clients.

7.4.2. Mapped Segments

Virtual address spaces in Amoeba are constructed from segments. When a process is started, it must have at least one segment. However, once it is running, a process can create additional segments and map them into its address space at any unused virtual address. Figure 7-7 shows a process with three memory segments currently mapped in.

A process can also unmap segments. Furthermore, a process can specify a range of virtual addresses and request that the range be unmapped, after which those addresses are no longer legal. When a segment or a range of addresses is unmapped, a capability is returned so the segment may still be accessed, or even mapped back in again later, possibly at a different virtual address.

A segment may be mapped into the address space of two or more processes at the same time. This allows processes to operate on shared memory. However, usually it is better to create a single process with multiple threads when shared memory is needed. The main reason for having distinct processes is better protection, but if the two processes are sharing memory, protection is generally not desired.

7.5. COMMUNICATION IN AMOEBA

Amoeba supports two forms of communication: RPC, using point-to-point message passing, and group communication. At the lowest level, an RPC consists of a request message followed by a reply message. Group communication uses hardware broadcasting or multicasting if it is available; otherwise, the kernel transparently simulates it with individual messages. In this section we will describe both Amoeba RPC and Amoeba group communication and then discuss the underlying FLIP protocol that is used to support them.

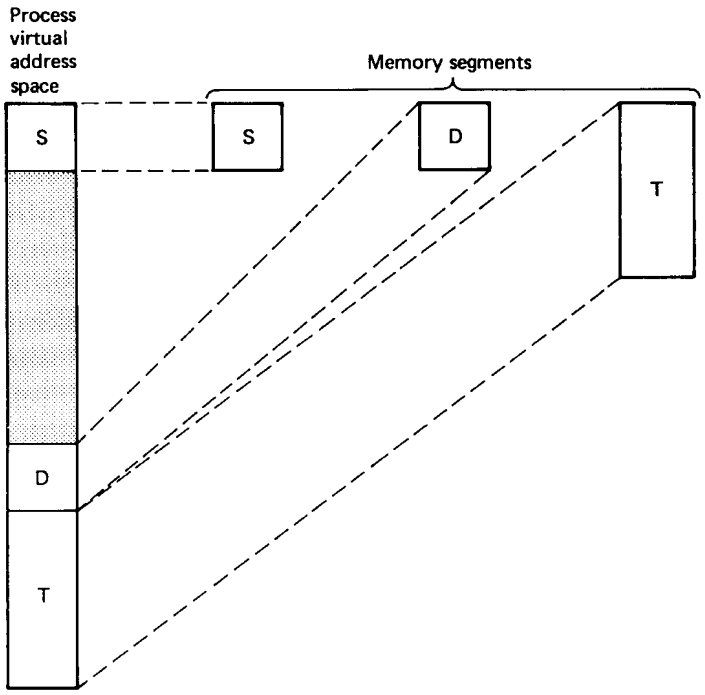


Fig. 7-7. A process with three segments mapped into its virtual address space.

7.5.1. Remote Procedure Call

Normal point-to-point communication in Amoeba consists of a client sending a message to a server followed by the server sending a reply back to the client. It is not possible for a client just to send a message and then go do something else except by bypassing the RPC interface, which is done only under very special circumstances. The RPC primitive that sends the request automatically blocks the caller until the reply comes back, thus forcing a certain amount of structure on programs. Separate *send* and *receive* primitives can be thought of as the distributed system's answer to the *goto* statement: parallel spaghetti programming. They should be avoided by user programs and used only by language runtime systems that have unusual communication requirements.

Each standard server defines a procedural interface that clients can call. These library routines are stubs that pack the parameters into messages and invoke the kernel primitives to send the message. During message transmission, the stub, and hence the calling thread, are blocked. When the reply comes back, the stub returns the status and results to the client. Although the kernel-level primitives are actually related to the message passing, the use of stubs makes

this mechanism look like RPC to the programmer, so we will refer to the basic communication primitives as RPC, rather than the slightly more precise "request/reply message exchange."

In order for a client thread to do an RPC with a server thread, the client must know the server's address. Addressing is done by allowing any thread to choose a random 48-bit number, called a **port**, to be used as the address for messages sent to it. Different threads in a process may use different ports if they so desire. All messages are addressed from a sender to a destination port. A port is nothing more than a kind of logical thread address. There is no data structure and no storage associated with a port. It is similar to an IP address or an Ethernet address in that respect, except that it is not tied to any particular physical location. The first field in each capability gives the port of the server that manages the object (see Fig. 7-3).

RPC Primitives

The RPC mechanism makes use of three principal kernel primitives:

1. get_request — indicates a server's willingness to listen on a port.
2. put_reply — done by a server when it has a reply to send.
3. trans — send a message from client to server and wait for the reply.

The first two are used by servers. The third is used by clients to *transmit* a message and wait for a reply. All three are true system calls, that is, they do not work by sending a message to a communication server thread. (If processes are able to send messages, why should they have to contact a server for the purpose of sending a message?) Users access the calls through library procedures, as usual, however.

When a server wants to go to sleep waiting for an incoming request, it calls *get_request*. This procedure has three parameters, as follows:

```
get_request(&header, buffer, bytes)
```

The first parameter points to a message header, the second points to a data buffer, and the third tells how big the data buffer is. This call is analogous to

```
read(fd, buffer, bytes)
```

in UNIX or MS-DOS in that the first parameter identifies what is being read, the second provides a buffer in which to put the data, and the third tells how big the buffer is.

When a message is transmitted over the network, it contains a header and (optionally) a data buffer. The header is a fixed 32-byte structure and is shown

in Fig. 7-8. What the first parameter of the *get_request* call does is tell the kernel where to put the incoming header. In addition, prior to making the *get_request* call, the server must initialize the header's *Port* field to contain the port it is listening to. This is how the kernel knows which server is listening to which port. The incoming header overwrites the one initialized by the server.

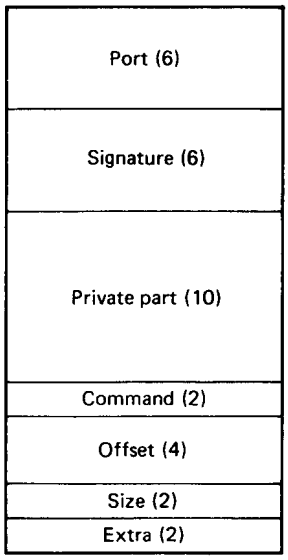


Fig. 7-8. The header used on all Amoeba request and reply messages. The numbers in parentheses give the field sizes in bytes.

When a message arrives, the server is unblocked. It normally first inspects the header to find out more about the request. The *Signature* field has been reserved for authentication purposes, but is not currently used.

The remaining fields are not specified by the RPC protocol, so a server and client can agree to use them any way they want. The normal conventions are as follows. Most requests to servers contain a capability, to specify the object being operated on. Many replies also have a capability as a return value. The *Private part* is normally used to hold the rightmost three fields of the capability.

Most servers support multiple operations on their objects, such as reading, writing, and destroying. The *Command* field is conventionally used on requests to indicate which operation is needed. On replies it tells whether the operation was successful or not, and if not, it gives the reason for failure.

The last three fields hold parameters, if any. For example, when reading a segment or file, they can be used to indicate the offset within the object to begin reading at, and the number of bytes to read.

Note that for many operations, no buffer is needed or used. In the case of

reading again, the object capability, the offset, and the size all fit in the header. When writing, the buffer contains the data to be written. On the other hand, the reply to a READ contains a buffer, whereas the reply to a WRITE does not.

After the server has completed its work, it makes a call

```
put_reply(&header, buffer, bytes)
```

to send back the reply. The first parameter provides the header and the second provides the buffer. The third tells how big the buffer is. If a server does a *put_reply* without having previously done an unmatched *get_request*, the *put_reply* fails with an error. Similarly, two consecutive *get_request* calls fail. The two calls must be paired in the correct way.

Now let us turn from the server to the client. To do an RPC, the client calls a stub which makes the following call:

```
trans(&header1, buffer1, bytes1, &header2, buffer2, bytes2)
```

The first three parameters provide information about the header and buffer of the outgoing request. The last three provide the same information for the incoming reply. The *trans* call sends the request and blocks the client until the reply has come in. This design forces processes to stick closely to the client-server RPC communication paradigm, analogous to the way structured programming techniques prevent programmers from doing things that generally lead to poorly structured programs (such as using unconstrained GOTO statements).

If Amoeba actually worked as described above, it would be possible for an intruder to impersonate a server just by doing a *get_request* on the server's port. These ports are public after all, since clients must know them to contact the servers. Amoeba solves this problem cryptographically. Each port is actually a pair of ports: the **get-port**, which is private, known only to the server, and the **put-port**, which is known to the whole world. The two are related through a one-way function, F, according to the relation

$$\text{put-port} = F(\text{get-port})$$

The one-way function is in fact the same one as used for protecting capabilities, but need not be since the two concepts are unrelated.

When a server does a *get_request*, the corresponding put-port is computed by the kernel and stored in a table of ports being listened to. All *trans* requests use put-ports, so when a packet arrives at a machine, the kernel compares the put-port in the header to the put-ports in its table to see if any match. Since get-ports never appear on the network and cannot be derived from the publicly known put-ports, the scheme is secure. It is illustrated in Fig. 7-9 and described in more detail in (Tanenbaum et al., 1986).

Amoeba RPC supports at-most-once semantics. In other words, when an

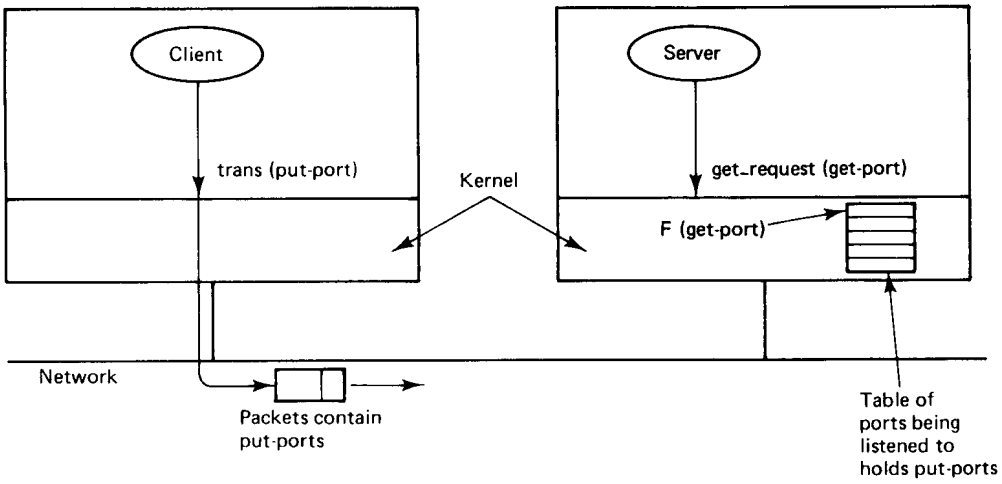


Fig. 7-9. Relationship between get-ports and put-ports.

RPC is done, the system guarantees that an RPC will never be carried out more than one time, even in the face of server crashes and rapid reboots.

7.5.2. Group Communication in Amoeba

RPC is not the only form of communication supported by Amoeba. It also supports group communication. A group in Amoeba consists of one or more processes that are cooperating to carry out some task or provide some service. Processes can be members of several groups at the same time. Groups are closed, meaning that only members can broadcast to the group. The usual way for a client to access a service provided by a group is to do an RPC with one of its members. That member then uses group communication within the group, if necessary, to determine who will do what.

This design was chosen to provide a greater degree of transparency than an open group structure would have. The idea behind it is that clients normally use RPC to talk to individual servers, so they should use RPC to talk to groups as well. The alternative—open groups and using RPC to talk to single servers but using group communication to talk to group servers—is much less transparent. (Using group communication for everything would eliminate the many advantages of RPC that we have discussed earlier.) Once it has been determined that clients outside a group will use RPC to talk to the group (actually, to talk to one process in the group), the need for open groups vanishes, so closed groups, which are easier to implement, are adequate.

Group Communication Primitives

The operations available for group communication in Amoeba are listed in Fig. 7-10. *CreateGroup* creates a new group and returns a group identifier used in the other calls to identify which group is meant. The parameters specify various sizes and how much fault tolerance is required (how many dead members the group must be able to withstand and continue to function correctly).

Call	Description
CreateGroup	Create a new group and set its parameters
JoinGroup	Make the caller a member of a group
LeaveGroup	Remove the caller from a group
SendToGroup	Reliably send a message to all members of a group
ReceiveFromGroup	Block until a message arrives from a group
ResetGroup	Initiate recovery after a process crash

Fig. 7-10. Amoeba group communication primitives.

JoinGroup and *LeaveGroup* allow processes to enter and exit from existing groups. One of the parameters of *JoinGroup* is a small message that is sent to all group members to announce the presence of a newcomer. Similarly, one of the parameters of *LeaveGroup* is another small message sent to all members to say goodbye and wish them good luck in their future activities. The point of the little messages is to make it possible for all members to know who their comrades are, in case they are interested, for example, to reconstruct the group if some members crash. When the last member of a group calls *LeaveGroup*, it turns out the lights and the group is destroyed.

SendToGroup atomically broadcasts a message to all members of a specified group, in spite of lost messages, finite buffers, and processor crashes. Amoeba supports global time ordering, so if two processes call *SendToGroup* simultaneously, the system ensures that all group members will receive the messages in the same order. This is guaranteed; programmers can count on it. If the two calls are exactly simultaneous, the first one to get its packet onto the LAN successfully is declared to be first. In terms of the semantics discussed in Chap. 6, this model corresponds to sequential consistency, not strict consistency.

ReceiveFromGroup tries to get a message from a group specified by one of its parameter. If no message is available (that is, currently buffered by the kernel), the caller blocks until one is available. If a message has already arrived,

the caller gets the message with no delay. The protocol ensures that in the absence of processor crashes, no messages are irretrievably lost. The protocol can also be made to tolerate crashes, at the cost of additional overhead, as discussed later.

The final call, *ResetGroup*, is used to recover from crashes. It specifies how many members the new group must have as a minimum. If the kernel is able to establish contact with the requisite number of processes and rebuild the group, it returns the size of the new group. Otherwise, it fails. In this case, recovery is up to the user program.

The Amoeba Reliable Broadcast Protocol

Let us now look at how Amoeba implements group communication. Amoeba works best on LANs that support either multicasting or broadcasting (or like Ethernet, both). For simplicity, we will just refer to broadcasting, although in fact the implementation uses multicasting when it can to avoid disturbing machines that are not interested in the message being sent. It is assumed that the hardware broadcast is good, but not perfect. In practice, lost packets are rare, but receiver overruns do happen occasionally. Since these errors can occur they cannot simply be ignored, so the protocol has been designed to deal with them.

The key idea that forms the basis of the implementation of group communication is **reliable broadcasting**. By this we mean that when a user process broadcasts a message (e.g., with *SendToGroup*) the user-supplied message is delivered correctly to all members of the group, even though the hardware may lose packets. For simplicity, we will assume that each message fits into a single packet. For the moment, we will assume that processors do not crash. We will consider the case of unreliable processors afterward. The description given below is just an outline. For more details, see (Kaashoek and Tanenbaum, 1991; and Kaashoek et al., 1989). Other reliable broadcast protocols are discussed in (Birman and Joseph, 1987a; Chang and Maxemchuk, 1984; Garcia-Molina and Spauster, 1991; Luan and Gligor, 1990; Melliar-Smith et al., 1990; and Tseung, 1989).

The hardware/software configuration required for reliable broadcasting in Amoeba is shown in Fig. 7-11. The hardware of all the machines is normally identical, and they all run exactly the same kernel. However, when the application starts up, one of the machines is elected as sequencer (like a committee electing a chairman). If the sequencer machine subsequently crashes, the remaining members elect a new one. Many election algorithms are known, such as choosing the process with the highest network address. We will discuss fault tolerance later in this chapter.

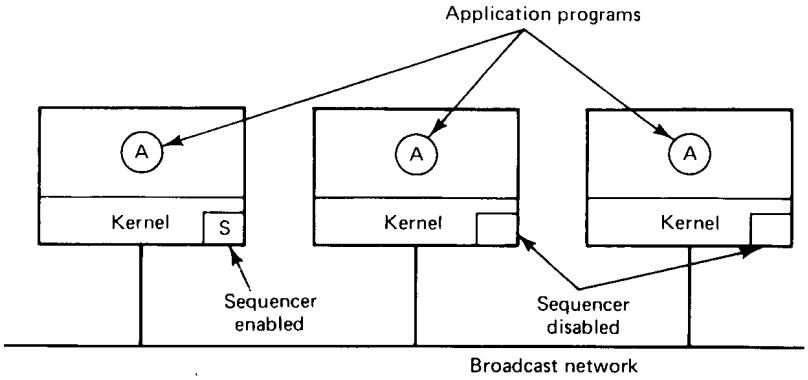


Fig. 7-11. System structure for group communication in Amoeba.

One sequence of events that can be used to achieve reliable broadcasting can be summarized as follows.

1. The user process traps to the kernel, passing it the message.
2. The kernel accepts the message and blocks the user process.
3. The kernel sends a point-to-point message to the sequencer.
4. When the sequencer gets the message, it allocates the next available sequence number, puts the sequence number in a header field reserved for it, and broadcasts the message (and sequence number).
5. When the sending kernel sees the broadcast message, it unblocks the calling process to let it continue execution.

Let us now consider these steps in more detail. When an application process executes a broadcasting primitive, such as *SendToGroup*, a trap to its kernel occurs. The kernel then blocks the caller and builds a message containing a kernel-supplied header and the application-supplied data. The header contains the message type (*Request for Broadcast* in this case), a unique message identifier (used to detect duplicates), the number of the last broadcast received by the kernel (usually called a **piggybacked acknowledgement**), and some other information.

The kernel sends the message to the sequencer using a normal point-to-point message, and simultaneously starts a timer. If the broadcast comes back before the timer runs out (normal case), the sending kernel stops the timer and returns control to the caller. In practice, this case happens well over 99 percent of the time, because modern LANs are highly reliable.

On the other hand, if the broadcast has not come back before the timer expires, the kernel assumes that either the message or the broadcast has been lost. Either way, it retransmits the message. If the original message was lost, no harm has been done, and the second (or subsequent) attempt will trigger the broadcast in the usual way. If the message got to the sequencer and was broadcast, but the sender missed the broadcast, the sequencer will detect the retransmission as a duplicate (from the message identifier) and just tell the sender that everything is all right. The message is not broadcast a second time.

A third possibility is that a broadcast comes back before the timer runs out, but it is the wrong broadcast. This situation arises when two processes attempt to broadcast simultaneously. One of them, *A*, gets to the sequencer first, and its message is broadcast. *A* sees the broadcast and unblocks its application program. However its competitor, *B*, sees *A*'s broadcast and realizes that it has failed to go first. Nevertheless, *B* knows that its message probably got to the sequencer (since lost messages are rare), where it will be queued and broadcast next. Thus *B* accepts *A*'s broadcast and continues to wait for its own broadcast to come back or its timer to expire.

Now consider what happens at the sequencer when a *Request for Broadcast* arrives there. First a check is made to see if the message is a retransmission, and if so, the sender is informed that the broadcast has already been done, as mentioned above. If the message is new (normal case), the next sequence number is assigned to it, and the sequencer counter incremented by 1 for next time. The message and its identifier are then stored in a **history buffer**, and the message is then broadcast. The message is also passed to the application running on the sequencer's machine (because the broadcast does not cause an interrupt on the machine that issued the broadcast).

Finally, let us consider what happens when a kernel receives a broadcast. First, the sequence number is compared to the sequence number of the broadcast received most recently. If the new one is 1 higher (normal case), no broadcasts have been missed, so the message is passed up to the application program, assuming that it is waiting. If it is not waiting, it is buffered until the program calls *ReceiveFromGroup*.

Suppose that the newly received broadcast has sequence number 25, while the previous one had number 23. The kernel is immediately alerted to the fact that it has missed number 24, so it sends a point-to-point message to the sequencer asking for a private retransmission of the missing message. The sequencer fetches the missing message from its history buffer and sends it. When it arrives, the receiving kernel processes 24 and 25, passing them to the application program in numerical order. Thus the only effect of a lost message is a (normally) minor time delay. All application programs see all broadcasts in the same order, even if some messages are lost.

The reliable broadcast protocol is illustrated in Fig. 7-12. Here the

application program running on machine *A* passes a message, *M*, to its kernel for broadcasting. The kernel sends the message to the sequencer, where it is assigned sequence number 25. The message (containing the sequence number 25) is now broadcast to all machines and also passed to the application program running on the sequencer itself. This broadcast message is denoted by *M25* in the figure.

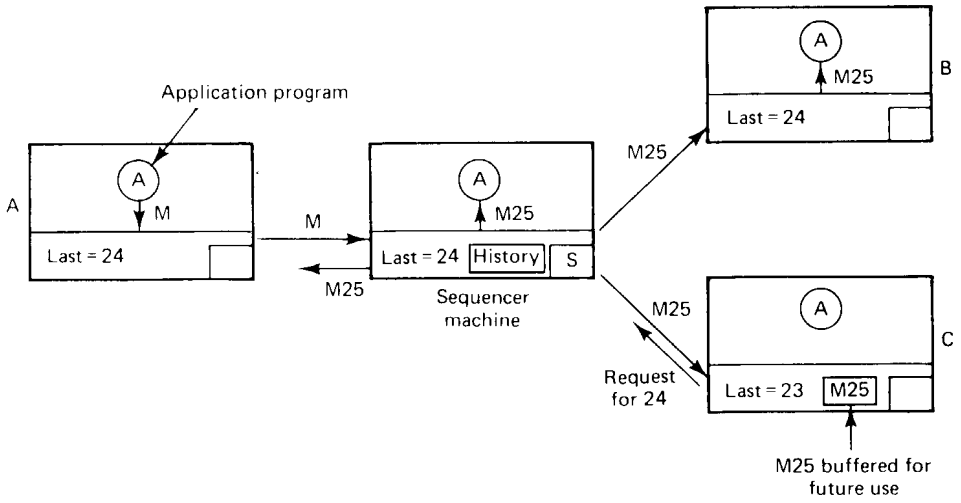


Fig. 7-12. The application of machine *A* sends a message to the sequencer, which then adds a sequence number (25) and broadcasts it. At *B* it is accepted, but at *C* it is buffered until 24, which was missed, can be retrieved from the sequencer.

The *M25* message arrives at machines *B* and *C*. At machine B the kernel sees that it has already processed all broadcasts up to and including 24, so it immediately passes *M25* up to the application program. At *C*, however, the last message to arrive was 23 (24 must have been lost), so *M25* is buffered in the kernel, and a point-to-point message requesting 24 is sent to the sequencer. Only after the reply has come back and been given to the application program will *M25* be passed upward as well.

Now let us look at the management of the history buffer. Unless something is done to prevent it, the history buffer will quickly fill up. However, if the sequencer knows that all machines have received broadcasts, say, 0 through 23, correctly, it can delete these from its history buffer.

Several mechanisms are provided to allow the sequencer to discover this information. The basic one is that each *Request for Broadcast* message sent to the sequencer carries a piggybacked acknowledgement, *k*, meaning that all broadcasts up to and including *k* have been correctly received. This way, the

sequencer can maintain a piggyback table, indexed by machine number, telling for each machine which broadcast was the last one received. Whenever the history buffer begins to fill up, the sequencer can make a pass through this table to find the smallest value. It can then safely discard all messages up to and including this value.

If one machine happens to be silent for an unusually long period of time, the sequencer will not know what its status is. To inform the sequencer, it is required to send a short acknowledgement message when it has sent no broadcast messages for a certain period of time. Furthermore, the sequencer can broadcast a *Request for Status* message, which directs all other machines to send it a message giving the number of the highest broadcast received in sequence. In this way, the sequencer can update its piggyback table and then truncate its history buffer.

Although in practice *Request for Status* messages are rare, they do occur, and thus raise the mean number of messages required for a reliable broadcast slightly above 2, even when there are no lost messages. The effect increases slightly as the number of machines grows.

There is a subtle design point concerning this protocol that should be clarified. There are two ways to do the broadcast. In method 1 (described above), the user sends a point-to-point message to the sequencer, which then broadcasts it. In method 2, the user broadcasts the message, including a unique identifier. When the sequencer sees this, it broadcasts a special *Accept* message containing the unique identifier and its newly assigned sequence number. A broadcast is "official" only when the *Accept* message has been sent. The two methods are compared in Fig. 7-13.

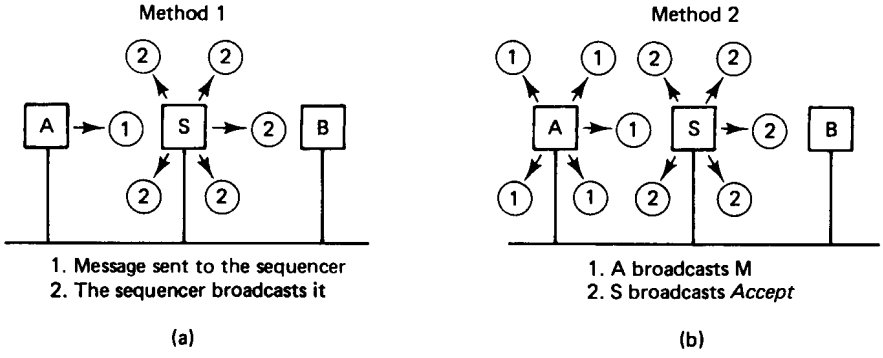


Fig. 7-13. Two methods for doing reliable broadcasting.

These protocols are logically equivalent, but they have different performance characteristics. In method 1, each message appears in full on the network twice: once to the sequencer and once from the sequencer. Thus a message of

length m bytes consumes $2m$ bytes worth of network bandwidth. However, only the second of these is broadcast, so each user machine is interrupted only once (for the second message).

In method 2, the full message appears only once on the network, plus a very short *Accept* message from the sequencer, so only half the bandwidth is consumed. On the other hand, every machine is interrupted twice, once for the message and once for the *Accept*. Thus method 1 wastes bandwidth to reduce interrupts compared to method 2. Depending on the average message size, one may be preferable to the other.

In summary, this protocol allows reliable broadcasting to be done on an unreliable network in just over two messages per reliable broadcast. Each broadcast is indivisible, and all applications receive all messages in the same order, no matter how many are lost. The worst that can happen is that some delay is introduced when a message is lost, which rarely happens. If two processes attempt to broadcast at the same time, one of them will get to the sequencer first and win. The other will see a broadcast from its competitor coming back from the sequencer, and will realize that its request has been queued and will appear shortly, so it simply waits.

Fault-Tolerant Group Communication

So far we have assumed that no processors crash. In fact, this protocol has been designed to withstand the loss of an arbitrary collection of k processors (including the sequencer), where k (the resilience degree) is selected when the group is created. The larger k is, the more redundancy is required, and the slower the operation is in the normal case, so the user must choose k with care. We will sketch the recovery algorithm below. For more details, see (Kaashoek and Tanenbaum, 1991).

When a processor crashes, initially no one detects this event. Sooner or later, however, some kernel notices that messages sent to the crashed machine are not being acknowledged, so the kernel marks the crashed processor as dead and the group as unusable. All subsequent group communication primitives on that machine fail (return an error status) until the group has been reconstructed.

Shortly after noticing a problem, the user process getting the error return calls *ResetGroup* to initiate recovery. The recovery is done in two phases (Garcia-Molina, 1982). In phase one, one process is elected as coordinator. In phase two, the coordinator rebuilds the group and brings all the other processes up to date. At that point, normal operation continues.

In Fig. 7-14(a) we see a group of six machines, of which machine 5, the sequencer, has just crashed. The numbers in the boxes indicate the last message correctly received by each machine. Two machines, 0 and 1, detect the sequencer failure simultaneously, and both call *ResetGroup* to start recovery.

This call results in the kernel sending a message to all other members inviting them to participate in the recovery and asking them to report back the sequence number of the highest message they have seen. At this point it is discovered that two processes have declared themselves coordinator. The one that has seen the message with the highest sequence number wins. In case of a tie, the one with the highest network address wins. This leads to a single coordinator, as shown in Fig. 7-14(b).

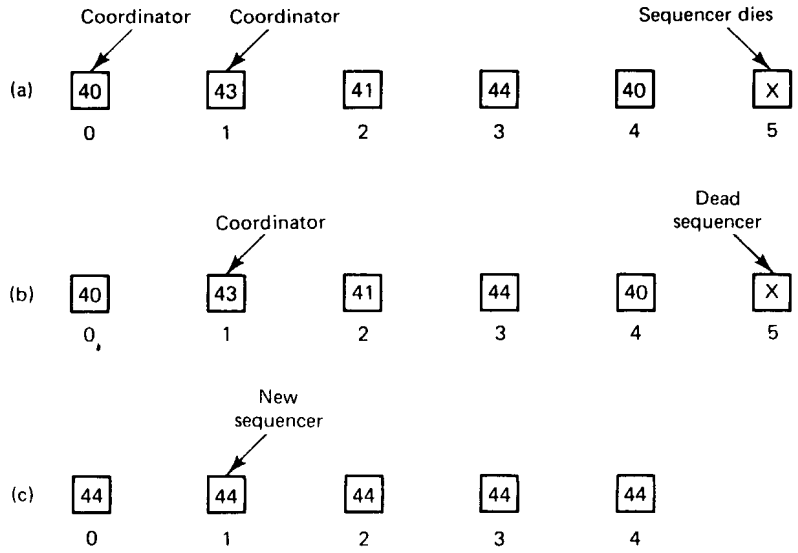


Fig. 7-14. (a) The sequencer crashes. (b) A coordinator is selected. (c) Recovery.

Once the coordinator has been voted into office, it collects from the other members any messages it may have missed. Now it is up to date and is able to become the new sequencer. It builds a *Results* message announcing itself as sequencer and telling the others what the highest sequence number is. Each member can now ask for any messages that it missed. When a member is up to date, it sends an acknowledgement back to the new sequencer. When the new sequencer has an acknowledgement from all the surviving members, it knows that all messages have been correctly delivered to the application programs in order, so it discards its history buffer, and normal operation can resume.

Another problem remains: How does the coordinator get any messages it has missed if the sequencer has crashed? The solution lies in the value of k, the resilience degree, chosen at group creation time. When k is 0 (non-fault tolerant case), only the sequencer maintains a history buffer. However, when k is greater than 0, $k + 1$ machines continuously maintain an up-to-date history buffer. Thus if an arbitrary collection of k machines fail, it is guaranteed that at least one

history buffer survives, and it is this one that supplies the coordinator with any messages it needs. The extra machines can maintain their history buffers simply by watching the network.

There is one additional problem that must be solved. Normally, a *SendToGroup* terminates successfully when the sequencer has received and broadcast or approved the message. If $k > 0$, this protocol is insufficient to survive k arbitrary crashes. Instead, a slightly modified version of method 2 is used. When the sequencer sees a message, M, that was just broadcast, it does not immediately broadcast an *Accept* message, as it does when $k = 0$. Instead, it waits until the k lowest-numbered kernels have acknowledged that they have seen and stored it. Only then does the sequencer broadcast the *Accept* message. Since $k + 1$ machines (including the sequencer) now are known to have stored M in their history buffers, even if k machines crash, M will not be lost.

As in the usual case, no kernel may pass M up to its application program until it has seen the *Accept* message. Because the *Accept* message is not generated until it is certain that $k + 1$ machines have stored M, it is guaranteed that if one machine gets M, they all will eventually. In this way, recovery from the loss of any k machines is always possible. As an aside, to speed up operation for $k > 0$, whenever an entry is made in a history buffer, a short control packet is broadcast to announce this event to the world.

To summarize, the Amoeba group communication scheme guarantees atomic broadcasting with global time ordering even in the face of k arbitrary crashes, where k is chosen by the user when the group is created. This mechanism provides an easy-to-understand basis for doing distributed programming. It is used in Amoeba to support object-based distributed shared memory for the Orca programming language and for other facilities. It can also be implemented efficiently. Measurements with 68030 CPUs on a 10-Mbps Ethernet show that it is possible to handle 800 reliable broadcasts per second continuously (Tanenbaum et al., 1992).

7.5.3. The Fast Local Internet Protocol

Amoeba uses a custom protocol called **FLIP** (**Fast Local Internet Protocol**) for actual message transmission. This protocol handles both RPC and group communication and is below them in the protocol hierarchy. In OSI terms, FLIP is a network layer protocol, whereas RPC is more of a connectionless transport or session protocol (the exact location is arguable, since OSI was designed for connection-oriented networks). Conceptually, FLIP can be replaced by another network layer protocol, such as IP, although doing so would cause some of Amoeba's transparency to be lost. Although FLIP were designed in the context of Amoeba, it is intended to be useful in other operating systems as well. In this section we will describe its design and implementation.

Protocol Requirements for Distributed Systems

Before getting into the details of FLIP, it is useful to understand something about why it was designed. After all, there are plenty of existing protocols, so the invention of a new one clearly has to be justified. In Fig. 7-15 we list the principal requirements that a protocol for a distributed system should meet. First, the protocol must support both RPC and group communication efficiently. If the underlying network has hardware multicast or broadcast, as Ethernet does, for example, the protocol should use it for group communication. On the other hand, if the network does not have either of these features, group communication must still work exactly the same way, even though the implementation will have to be different.

Item	Description
RPC	The protocol should support RPC
Group communication	The protocol should support group communication
Process migration	Processes should be able to take their addresses with them
Security	Processes should not be able to impersonate other processes
Network management	Support is needed for automatic reconfiguration
Wide-area networks	The protocol should also work on wide area networks

Fig. 7-15. Desirable characteristics for a distributed system protocol.

A characteristic that is increasingly important is support for process migration. A process should be able to move from one machine to another, even to one in a different network, with nobody noticing. Protocols such as OSI, X.25, and TCP/IP that use machine addresses to identify processes make migration difficult, because a process cannot take its address with it when it moves.

Security is also an issue. Although the get-ports and put-ports provide security for Amoeba, a security mechanism should also be present in the packet protocol so it can be used with operating systems that do not have cryptographically secure addresses.

Another point on which most existing protocols score badly is network management. It should not be necessary to have elaborate configuration tables telling which network is connected to which other network. Furthermore, if the configuration changes, due to gateways going down or coming back up, the protocol should adapt to the new configuration automatically.

Finally, the protocol should work on both local and wide-area networks. In particular, the same protocol should be usable on both.

The FLIP Interface

The FLIP protocol and its associated architecture was designed to meet all these requirements. A typical FLIP configuration is shown in Fig. 7-16. Here we see five machines, two on an Ethernet and four on a token ring. Each machine has one user process, *A* through *E*. One of the machines is connected to both networks, and as such, functions automatically as a gateway. Gateways may also run clients and servers, just like other nodes.

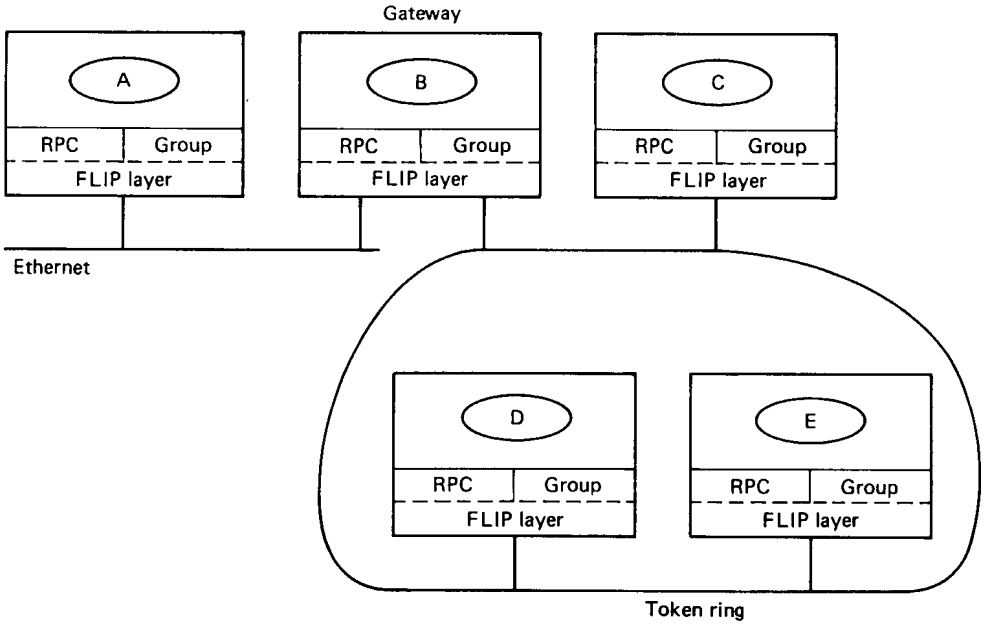


Fig. 7-16. A FLIP system with five machines and two networks.

The software is structured as shown in Fig. 7-16. The kernel contains two layers. The top layer handles calls from user processes for RPC or group communication services. The bottom layer handles the FLIP protocol. For example, when a client calls *trans*, it traps to the kernel. The RPC layer examines the header and buffer, builds a message from them, and passes the message down to the FLIP layer for transmission.

All low-level communication in Amoeba is based on **FLIP addresses**. Each process has exactly one FLIP address: a 64-bit random number chosen by the system when the process is created. If the process ever migrates, it takes its FLIP address with it. If the network is ever reconfigured, so that all machines are assigned new (hardware) network numbers or network addresses, the FLIP addresses still remain unchanged. It is the fact that a FLIP address uniquely

identifies a process, not a machine, that makes communication in Amoeba insensitive to changes in network topology and network addressing.

A FLIP address is really two addresses, a public-address and a private-address, related by

$$\text{Public-address} = \text{DES}(\text{private-address})$$

where DES is the Data Encryption Standard. To compute the public-address from the private one, the private-address is used as a DES key to encrypt a 64-bit block of 0s. Given a public-address, finding the corresponding private address is computationally infeasible. Servers listen to private-addresses, but clients send to public-addresses, analogous to the way put-ports and get-ports work, but at a lower level.

FLIP has been designed to work not only with Amoeba, but also with other operating systems. A version for UNIX also exists, and there is no reason one could not be made for MS-DOS. The security provided by the private-address, public-address scheme also works for UNIX to UNIX communication using FLIP, independent of Amoeba.

Furthermore, FLIP has been designed so that it can be built in hardware, for example, as part of the network interface chip. For this reason, a precise interface with the layer above it has been specified. The interface between the FLIP layer and the layer above it (which we will call the RPC layer) has nine primitives, seven for outgoing traffic and two for incoming traffic. Each one has a library procedure that invokes it. The nine calls are listed in Fig. 7-17.

	Description	Direction
Init	Allocate a table slot	↓
End	Return a table slot	↓
Register	Listen to a FLIP address	↓
Unregister	Stop listening	↓
Unicast	Send a point-to-point message	↓
Multicost	Send a multicost message	↓
Broadcast	Send a broadcast message	↓
Receive	Packet received	↑
Notdeliver	Undeliverable packet received	↑

Fig. 7-17. The calls supported by the FLIP layer.

The first one, *init*, allows the RPC layer to allocate a table slot and initialize it with pointers to two procedures (or in a hardware implementation, two interrupt vectors). These procedures are the ones called when normal and undeliverable packets arrive, respectively. *End* deallocates the slot when the machine is being shut down.

Register is invoked to announce a process' FLIP address to the FLIP layer. It is called when the process starts up (or at least, on the first attempt at getting or sending a message). The FLIP layer immediately runs the private-address offered to it through the DES function and stores the public-address in its tables. If an incoming packet is addressed to the public FLIP address, it will be passed to the RPC layer for delivery. The *unregister* call removes an entry from the FLIP layer's tables.

The next three calls are for sending point-to-point messages, multicast messages, and broadcast messages, respectively. None of these guarantee delivery. To make RPC reliable, acknowledgements are used. To make group communication reliable, even in the face of lost packets, the sequencer protocol discussed above is used.

The last two calls are for incoming traffic. The first is for messages originating elsewhere and directed to this machine. The second is for messages sent by this machine but sent back as undeliverable.

Although the FLIP interface is intended primarily for use by the RPC and broadcast layers within the kernel, it is also visible to user processes, in case they have a special need for raw communication.

Operation of the FLIP Layer

Packets passed by the RPC layer or the group communication layer (see Fig. 7-16) to the FLIP layer are addressed by FLIP addresses, so the FLIP layer must be able to convert these addresses to network addresses for actual transmission. In order to perform this function, the FLIP layer maintains the routing table shown in Fig. 7-18. Currently this table is maintained in software, but chip designers could implement it in hardware in the future.

Whenever an incoming packet arrives at any machine, it is first handled by the FLIP layer, which extracts from it the FLIP address and network address of the sender. The number of hops the packet has made is also recorded. Since the hop count is incremented only when a packet is forwarded by a gateway, the hop count tells how many gateways the packet has passed through. The hop count is therefore a crude attempt to measure how far away the source is. (Actually, things are slightly better than this, as slow networks can be made to count for multiple hops.) If the FLIP address is not presently in the routing table, it is entered. This entry can later be used to send packets *to* that FLIP address, since its network number and address are now known.

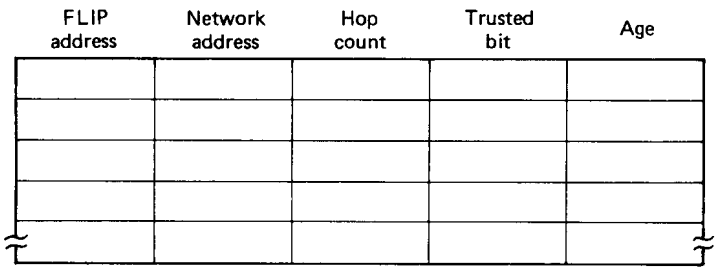


Fig. 7-18. The FLIP routing table.

An additional bit present in each packet tells whether the path the packet has followed so far is entirely over trusted networks. It is managed by the gateways. If the packet has gone through one or more untrusted networks, packets to the source address should be encrypted if absolute security is desired. With trusted networks, encryption is not needed.

The last field of each routing table entry gives the age of the routing table entry. It is reset to 0 whenever a packet is received from the corresponding FLIP address. Periodically, all the ages are incremented. This field allows the FLIP layer to find a suitable table entry to purge if the table fills up (large numbers indicate that there has been no traffic for a long time).

Locating Put-Ports

To see how FLIP works in the context of Amoeba, let us consider a simple example using the configuration of Fig. 7-16. *A* is a client and *B* is a server. With FLIP, any machine having connections to two or more networks is automatically a gateway, so the fact that *B* happens to be running on a gateway machine is irrelevant.

When *B* is created, the kernel picks a new random FLIP address for it and registers it with the FLIP layer. After starting, *B* initializes itself and then does a *get_request* on its get-port, which causes a trap to the kernel. The RPC layer looks up the put-port in its get-port to put-port cache (or computes it if no entry is found) and makes a note that a process is listening to that port. It then blocks until a request comes in.

Later, *A* does a *trans* on the put-port. Its RPC layer looks in its tables to see if it knows the FLIP address of the server process that listens to the put-port. Since it does not, the RPC layer sends a special broadcast packet to find it. This packet has a maximum hop count set to make sure that the broadcast is confined to its own network. (When a gateway sees a packet whose current hop count is already equal to its maximum hop count, the packet is discarded instead of being forwarded.) If the broadcast fails, the sending RPC layer times out and tries

again with a maximum hop count one larger, and so on, until it locates the server.

When the broadcast packet arrives at *B*'s machine, the RPC layer there sends back a reply announcing its FLIP address. Like all incoming packets, this packet causes *A*'s FLIP layer to make an entry for that FLIP address before passing the reply packet up to the RPC layer. The RPC layer now makes an entry in its own tables mapping the put-port onto the FLIP address. Then it sends the request to the server. Since the FLIP layer now has an entry for the server's FLIP address, it can build a packet containing the proper network address and send it without further ado. Subsequent requests to the server's put-port use the RPC layer's cache to find the FLIP address and the FLIP layer's routing table to find the network address. Thus broadcasting is used only the very first time a server is contacted. After that, the kernel tables provide the necessary information.

To summarize, locating a put-port requires two mappings:

1. From the put-port to the FLIP address (done by the RPC layer).
2. From the FLIP address to the network address (done by the FLIP layer).

The reason for this two-stage process is twofold. First, FLIP has been designed as a general-purpose protocol for use in distributed systems, including non-Amoeba systems. Since these systems generally do not use Amoeba-style ports, the mapping of put-ports to FLIP addresses has not been built into the FLIP layer. Other users of FLIP may just use FLIP addresses directly.

Second, a put-port really identifies a *service* rather than a *server*. A service may be provided by multiple servers to enhance performance and reliability. Although all the servers listen to the same put-port, each one has its own private FLIP address. When a client's RPC layer issues a broadcast to find the FLIP address corresponding to a put-port, any or all of the servers may respond. Since each server has a different FLIP address, each response creates a different routing table entry. All the responses are passed to the RPC layer, which chooses one to use.

The advantage of this scheme over having just a single (port, network address) cache is that it permits servers to migrate to new machines or have their machines be wheeled over to new networks and plugged in without requiring any manual reconfiguration, as, say, TCP/IP does. There is a strong analogy here with a person moving and being assigned the same telephone number at the new residence as he had at the old one. (For the record, Amoeba does not currently support process migration, but this feature could be added in the future.)

The advantage over having clients and servers use FLIP addresses directly is

the protection offered by the one-way function used to derive put-ports from get-ports. In addition, if a server crashes, it will pick a new FLIP address when it reboots. Attempts to use the old FLIP address will time out, allowing the RPC layer to indicate failure to the client. This mechanism is how at-most-once semantics are guaranteed. The client, however, can just try again with the same put-port if it wishes, since that is not necessarily invalidated by server crashes.

FLIP over Wide-Area Networks

FLIP also works transparently over wide-area networks. In Fig. 7-19 we have three local-area networks connected by a wide-area network. Suppose that the client *A* wants to do an RPC with the server *E*. *A*'s RPC layer first tries to locate the put-port using a maximum hop count of 1. When that fails, it tries again with a maximum hop count of 2. This time, *C* forwards the broadcast packet to all the gateways that are connected to the wide-area network, namely, *D* and *G*. Effectively, *C* simulates broadcast over the wide-area network by sending individual messages to all the other gateways. When this broadcast fails to turn up the server, a third broadcast is sent, this time with a maximum hop count of 3. This one succeeds. The reply contains *E*'s network address and FLIP address, which are then entered into *A*'s routing table. From this point on, communication between *A* and *E* happens using normal point-to-point communication. No more broadcasts are needed.

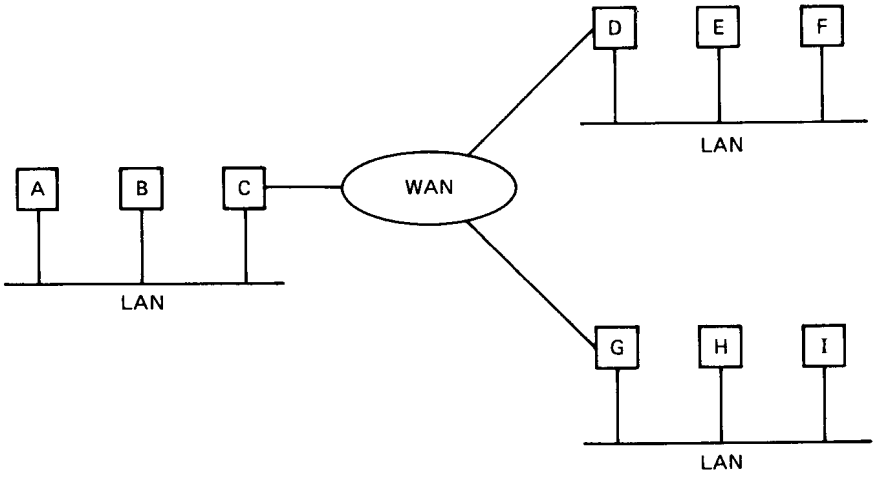


Fig. 7-19. Three LANs connected by a WAN.

Communication over the wide-area network is encapsulated in whatever protocol the wide-area network requires. For example, on a TCP/IP network, *C*

might have open connections to D and G all the time. Alternatively, the implementation might decide to close any connection not used for a certain length of time.

Although this method does not scale well to thousands of LANs, for modest numbers it works quite well. In practice, few servers move, so that once a server has been located by broadcasting, subsequent requests will use the cached entries. Using this method, a substantial number of machines all over the world can work together in a totally transparent way. An RPC to a thread in the caller's address space and an RPC to a thread halfway around the world are done in exactly the same way.

Group communication also uses FLIP. When a message is sent to multiple destinations, FLIP uses the hardware multicast or broadcast on those networks where it is available. On those that do not have it, broadcast is simulated by sending individual messages, just as we saw on the wide-area network. The choice of mechanism is done by the FLIP layer, with the same user semantics in all cases.

7.6. THE AMOEBA SERVERS

Most of the traditional operating system services (such as the file server) are implemented in Amoeba as server processes. Although it would have been possible to put together a random collection of servers, each with its own model of the world, it was decided early on to provide a single model of what a server does to achieve uniformity and simplicity. Although voluntary, most servers follow it. The model, and some examples of key Amoeba servers, are described in this section.

All standard servers in Amoeba are defined by a set of stub procedures. The newer stubs are defined in **AIL**, the **Amoeba Interface Language**, although the older ones are handwritten in C. The stub procedures are generated by the AIL compiler from the stub definitions and then placed in the library so that clients can use them. In effect, the stubs define precisely what services the server provides and what their parameters are. In our discussion below, we will refer to the stubs frequently.

7.6.1. The Bullet Server

Like all operating systems, Amoeba has a file system. However, unlike most other ones, the choice of file system is not dictated by the operating system. The file system runs as a collection of server processes. Users who do not like the standard ones are free to write their own. The kernel does not know, or

care, which is the "real" file system. In fact, different users may use different and incompatible file systems at the same time if they desire.

The standard file system consists of three servers, the **bullet server**, which handles file storage, the **directory server**, which takes care of file naming and directory management, and the **replication server**, which handles file replication. The file system has been split into these separate components to achieve increased flexibility and make each of the servers straightforward to implement. We will discuss the bullet server in this section and the other two in the following ones.

Very briefly, a client process can create a file using the *create* call. The bullet server responds by sending back a capability that can be used in subsequent calls to *read* to retrieve all or part of the file. In most cases, the user will then give the file an ASCII name, and the (ASCII name, capability) pair will be given to the directory server for storage in a directory, but this operation has nothing to do with the bullet server.

The bullet server was designed to be very fast (hence the name). It was also designed to run on machines having large primary memories and huge disks, rather than on low-end machines, where memory is always scarce. The organization is quite different from that of most conventional file servers. In particular, files are **immutable**. Once a file has been created, it cannot subsequently be changed. It can be deleted and a new file created in its place, but the new file has a different capability from the old one. This fact simplifies automatic replication, as will be seen. It is also well suited for use on large-capacity, write-once optical disks.

Because files cannot be modified after their creation, the size of a file is always known at creation time. This property allows files to be stored contiguously on the disk and also in the main memory cache. By storing files contiguously, they can be read into memory in a single disk operation, and they can be sent to users in a single RPC reply message. These simplifications lead to the high performance.

The conceptual model behind the file system is thus that a client creates an entire file in its own memory, and then transmits it in a single RPC to the bullet server, which stores it and returns a capability for accessing it later. To modify this file (e.g., to edit a program or document), the client sends back the capability and asks for the file, which is then (ideally) sent in one RPC to the client's memory. The client can then modify the file locally any way it wants to. When it is done, it sends the file to the server (ideally) in one RPC, thus causing a new file to be created and a new capability to be returned. At this point the client can ask the server to destroy the original file, or it can keep the old file as a backup.

As a concession to reality, the bullet server also supports clients that have too little memory to receive or send entire files in a single RPC. When reading, it is possible to ask for a section of a file, specified by an offset and a byte count.

This feature allows clients to read files in whatever size unit they find convenient.

Writing a file in several operations is complicated by the fact that bullet server files are guaranteed to be immutable. This problem is dealt with by introducing two kinds of files, **uncommitted files**, which are in the process of being created, and **committed files**, which are permanent. Uncommitted files can be changed; committed files cannot be. An RPC doing a *create* must specify whether the file is to be committed immediately or not.

In both cases, a copy of the file is made at the server and a capability for the file is returned. If the file is not committed, it can be modified by subsequent RPCs; in particular, it can be appended to. When all the appends and other changes have been completed, the file can be committed, at which point it becomes immutable. To emphasize the transient nature of uncommitted files, they cannot be read. Only committed files can be read.

The Bullet Server Interface

The bullet server supports the six operations listed in Fig. 7-20, plus an additional three that are reserved for the system administrator. In addition, the relevant standard operations listed in Fig. 7-5 are also valid. All these operations are accessed by calling stub procedures from the library.

Call	Description
Create	Create a new file; optionally commit it as well
Read	Read all or part of a specified file
Size	Return the size of a specified file
Modify	Overwrite n bytes of an uncommitted file
Insert	Insert or append n bytes to an uncommitted file
Delete	Delete n bytes from an uncommitted file

Fig. 7-20. Bullet server calls.

The *create* procedure supplies some data, which are put into a new file whose capability is returned in the reply. If the file is committed (determined by a parameter), it can be read but not changed. If it is not committed, it cannot be read until it is committed, but it can be changed or appended to.

The *read* call can read all or part of any committed file. It specifies the file to be read by providing a capability for it. Presentation of the capability is proof that the operation is allowed. The bullet server does not make any checks based

on the client's identity: it does not even know the client's identity. The *size* call takes a capability as parameter and tells how big the corresponding file is.

The last three calls all work on uncommitted files. They allow the file to be changed by overwriting, inserting, or deleting bytes. Multiple calls can be made in succession. The last call can indicate via a parameter that it wants to commit the file.

The bullet server also supports three special calls for the system administrator, who must present a special super-capability. These calls flush the main memory cache to disk, allow the disk to be compacted, and repair damaged file systems.

The capabilities generated and used by the bullet server use the *Rights* field to protect the operations. In this way, a capability can be made that allows a file to be read but not to be destroyed, for example.

Implementation of the Bullet Server

The bullet server maintains a file table with one entry per file, analogous to the UNIX i-node table and shown in Fig. 7-21. The entire table is read into memory when the bullet server is booted and is kept there as long as the bullet server is running.

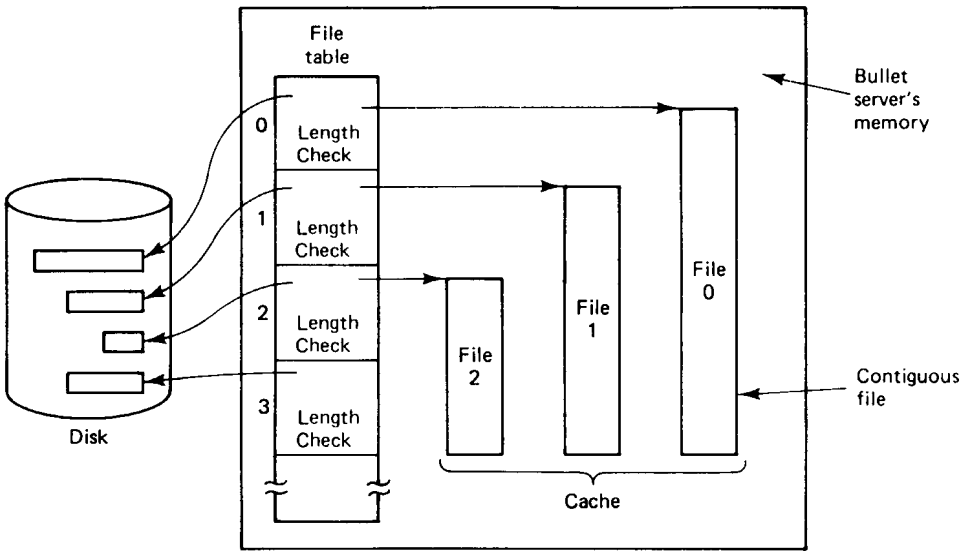


Fig. 7-21. Implementation of the bullet server.

Roughly speaking, each table entry contains two pointers and a length, plus some additional information. One pointer gives the disk address of the file and

the other gives the main memory address if the file happens to be in the main memory cache at the moment. All files are stored contiguously, both on disk and in the cache, so a pointer and a length is enough. Unlike UNIX, no direct or indirect blocks are needed.

Although this strategy wastes space due to external fragmentation, both in memory and on disk, it has the advantage of extreme simplicity and high performance. A file on disk can be read into memory in a single operation, at the maximum speed of the disk, and it can be transmitted over the network at the maximum speed of the network. As memories and disks get larger and cheaper, it is likely that the cost of the wasted memory will be acceptable in return for the speed provided.

When a client process wants to read a file, it sends the capability for the file to the bullet server. The server extracts the object number from the capability and uses it as an index into the file table to locate the entry for the file. The entry contains the random number used in the capability's *Check* field, which is then used to verify that the capability is valid. If it is invalid, the operation is terminated with an error code. If it is valid, the entire file is fetched from the disk into the cache, unless it is already there. Cache space is managed using LRU, but the implicit assumption is that the cache is usually large enough to hold the set of files currently in use.

If a file is created and the capability lost, the file can never be accessed but will remain forever. To prevent this situation, timeouts are used. An uncommitted file that has not been accessed in 10 minutes is simply deleted and its table entry freed. If the entry is subsequently reused for another file, but the old capability is presented 15 minutes later, the *Check* field will detect the fact that the file has changed and the operation on the old file will be rejected. This approach is acceptable because files normally exist in the uncommitted state for only a few seconds.

For committed files, a less draconian method is used. Associated with every file (in the file table entry) is a counter, initialized to *MAX_LIFETIME*. Periodically, a daemon does an RPC with the bullet server, asking it to perform the standard *age* operation (see Fig. 7-5). This operation causes the bullet server to run through the file table, decrementing each counter by 1. Any file whose counter goes to 0 is destroyed and its disk, table, and cache space reclaimed.

To prevent this mechanism from removing files that are in use, another operation, *touch*, is provided. Unlike *age*, which applies to all files, *touch* is for a specific file. Its function is to reset the counter to *MAX_LIFETIME*. *Touch* is called periodically for all files listed in any directory, to keep them from timing out. Typically, every file is touched once an hour, and a file is deleted if it has not been touched in 24 hours. This mechanism removes lost files (i.e., files not in any directory).

The bullet server can run in user space as an ordinary process. However, if

it is running on a dedicated machine, with no other processes on that machine, a small performance gain can be achieved by putting it in the kernel. The semantics are unchanged by this move. Clients cannot even tell where it is located.

7.6.2. The Directory Server

The bullet server, as we have seen, just handles file storage. The naming of files and other objects is handled by the **directory server**. Its primary function is to provide a mapping from human-readable (ASCII) names to capabilities. Processes can create one or more directories, each of which can contain multiple rows. Each row describes one object and contains both the object's name and its capability. Operations are provided to create and delete directories, add and delete rows, and look up names in directories. Unlike bullet files, directories are *not* immutable. Entries can be added to existing directories and entries can be deleted from existing directories.

Directories themselves are objects and are protected by capabilities, just as other objects. The operations on a directory, such as looking up names and adding new entries, are protected by bits in the *Rights* field, in the usual way. Directory capabilities may be stored in other directories, permitting hierarchical directory trees and more general structures.

Although the directory server can be used simply to store (file-name, capability) pairs, it can also support a more general model. First, a directory entry can name any kind of object that is described by a capability, not just a bullet file or directory. The directory server neither knows nor cares what kind of objects its capabilities control. The entries in a single directory may be for a variety of different kinds of objects, and these objects may be scattered randomly all over the world. There is no requirement that objects in a directory all be the same kind or all be managed by the same server. When a capability is fetched, its server is located by broadcasting, as described earlier.

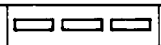
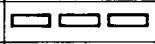
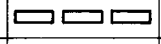
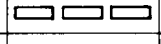
ASCII string	Capability set	Owner	Group	Others
Mail		1111	0000	0000
Games		1111	1110	1110
Exams		1111	0000	0000
Papers		1111	1100	1000
Committees		1111	1010	0010

Fig. 7-22. A typical directory managed by the directory server.

Second, a row may contain not just one capability, but a whole set of capabilities, as shown in Fig. 7-22. Generally, these capabilities are for identical